
CHAPTER 4

Experiments

En este capítulo se describen los modelos implementados y los experimentos realizados para entrenarlos y evaluarlos. Primero se explicaran los experimentos realizados como baseline. A continuación se explicará el proceso iterativo con el que se han realizado diferentes experimentos con el objetivo de encontrar un modelo de predicción y un set de datos óptimos.

4.1 Baseline forecasting models

Un modelo baseline es un modelo prototipo, muy simple, que sirve como referencia para evaluar el rendimiento de modelos más complejos. En este caso el modelo baseline recibe como input las cuotas de empate, home-win y away-win de la casa de apuestas *Bet365*. A partir de este input, el modelo deberá predecir el resultado de los partidos. Más tarde se compararan los resultados de modelos más complejos con este modelo, lo que servirá para ver si los nuevos modelos rinden como se espera, ya que si un nuevo modelo iguala o supera el acierto del baseline significará que tiene una mejor capacidad de predicción que las probabilidades que asignan las casas de apuestas.



Se han implementado diversos modelos baseline basados en diferentes algoritmos muy conocidos de Machine Learning, como Support Vector Machine (SVM), Random Forest (RF) o Multi-Layer Perceptron (MLP). Es importante destacar que las cuotas han sido transformadas a probabilidades (entre 0 y 1), simplemente haciendo la inversa de la cuota.



Baseline I: max odds Este primer baseline es el más sencillo. Simplemente consiste en la predicción que hace *Bet365*, ya que el modelo escoge la máxima probabilidad entre empate, home-win y away-win. Como ya mencionamos en el análisis de las cuotas (Figure 3.10) las casas de apuestas nunca predicen que el partido finalizará en empate, ya que siempre asignan una probabilidad mayor a la victoria de uno de los equipos. El acierto obtenido ha sido del 54.45%. La Figure 4.1 muestra la confusion matrix de los resultados y la curva AUC-ROC. Se aprecia en ambas figuras como este simple modelo tiene grandes dificultades a la hora de distinguir los empates, teniendo esta clase el Area Under Curve (AUC) más pequeño de las tres 0.61.

Baseline II: SVM  segundo baseline consiste en un modelo SVM entrenado con 3-fold Cross-Validation (CV) y los hiperparámetros se han optimizado con una búsqueda randomizada de hiperparámetros, usando la clase de *Scikit-Learn*, `model_selection.RandomizedSearchCV`.



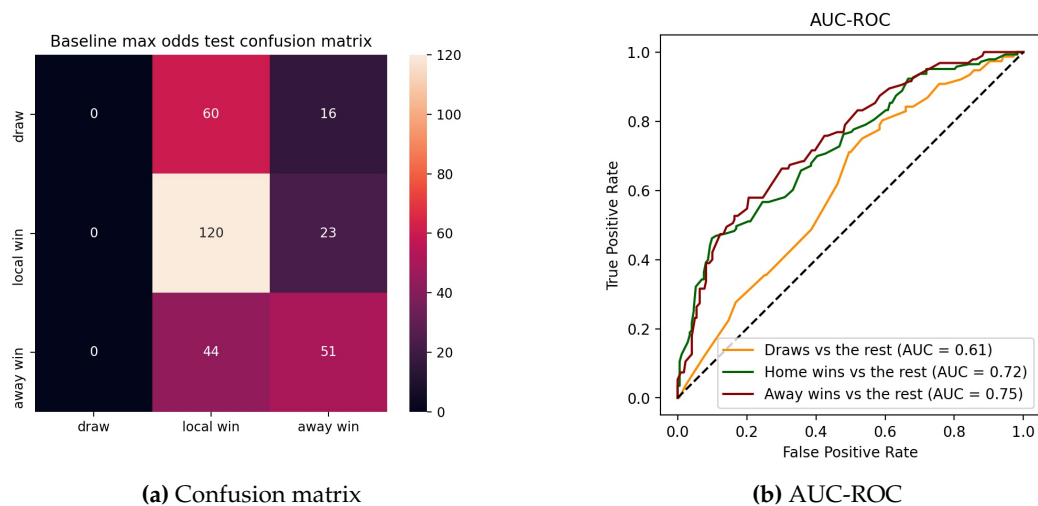


Figure 4.1: Confusion matrix and AUC-ROC curve of max-odds baseline.

Esta clase ya incorpora la opción del entrenamiento con CV. Los hiperparámetros optimizados han sido el hiperparámetro de regularización de SVM (C), el *kernel* usado en el SVM (linear, polynomial or rbf), el hiperparámetro γ , que es el kernel coefficient para los RBF-SVM y polynomial SVM, y, por último, el hiperparámetro $degree$ que indica el grado del polinomio en el kernel polinomial.

El modelo con mejores resultados ha obtenido un acierto en el test del 54.78% con $C = 2.99$, $kernel = rbf$ y $\gamma = 0.75$.

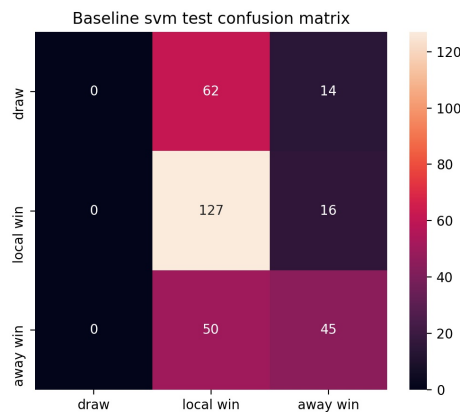


Figure 4.2: Test evaluation of Support Vector Machine based baseline confusion matrix.

Baseline III: Random Forest El baseline basado en un Random Forest también ha sido entrenado con 3-fold CV, y los hiperparámetros optimizados con random search han sido el número de estimadores (árboles de decisión), profundidad máxima de los árboles, el mínimo de samples necesarios para hacer un *split* en el árbol, el mínimo de samples para poder crear una hoja y el máximo de hojas que pueden tener los árboles. Todos estos parámetros se optimizan para evitar el overfitting y obtener la máximo acierto posible en el test.

El modelo Random Forest con mejores resultado ha obtenido un acierto en el test de 54.73%, con 112 árboles de decisión, 96 partidos mínimo para crear un *split*, 4 samples mínimo por hoja y un máximo de 5 hojas por árbol.

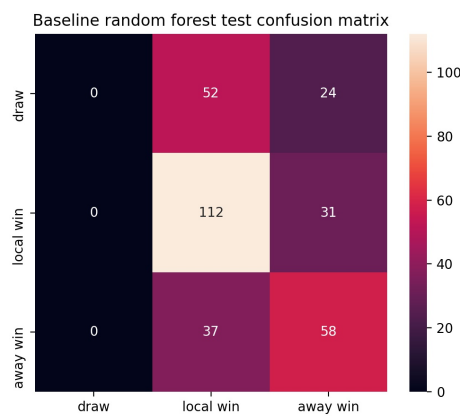


Figure 4.3: Test evaluation of Random Forest based baseline confusion matrix.

Baseline IV: MLP Por último se ha creado un baseline basado en Multi-layer Perceptron con una hidden layer de tres units. Como los anteriores modelos, también ha sido entrenado con 3-fold CV, y en este caso no se ha realizado una optimización de los hiperparámetros. Los hiperparámetros usados han sido Cross Entropy como función de error, Stochastic Gradient Descend (SGD) como optimizador con un learning rate de 0.1, 100 epochs y un batch-size de 32. Se ha obtenido un acierto del 54.78%, ligeramente superior a los demás modelos, excepto a SVM, que lo iguala.

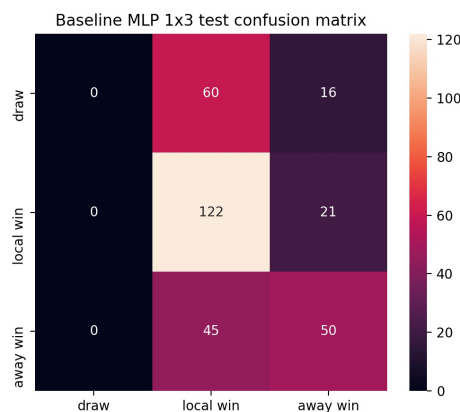


Figure 4.4: Test evaluation of Multi-Perceptron based baseline confusion matrix.

Estos resultados de porcentaje de acierto en el test trataran de ser superados por modelos de Machine Learning más complejos. Según las matrices de confusión, todos los modelos se comportan de una manera muy parecida, prediciendo mucho más victorias locales que el resto de resultados posibles, lo cual tiene sentido según la distribución que seguían los resultados. Sin embargo, es notable recalcar que ningún modelo predice ningún empate, posiblemente por lo desbalanceado que está el dataset (Table 3.9). Se muestra en la Figure 4.1 la AUC-ROC, donde vemos que el target de Empates es el que más le cuesta a

las Casas de apuestas de identificar. Será también, pues, objetivo de los modelos que se desarrollaran, el mejorar el acierto en la predicción de empates.

	Max-Odds	SVM	Random Forest	MLP
%	54.46	54.78	54.73	54.78

Table 4.1: Baseline models test accuracy.

4.2 Forecasting models

En esta sección se explican los experimentos llevados a cabo en los que se han entrenado los datasets creados, explicados en el capítulo anterior, con dos algoritmos de Machine Learning diferentes: Redes Neuronales y Gradient Boosting. Como se ha explicado en el capítulo de la **Metodología y la Implementación**, el enfoque que se ha llevado a cabo es empezar realizando experimentos con soluciones y modelos sencillos, y sobre ellos se ha ido construyendo la infraestructura software necesaria para analizar resultado e implementar algoritmos y soluciones cada vez más complejas si el análisis así lo indicaba.

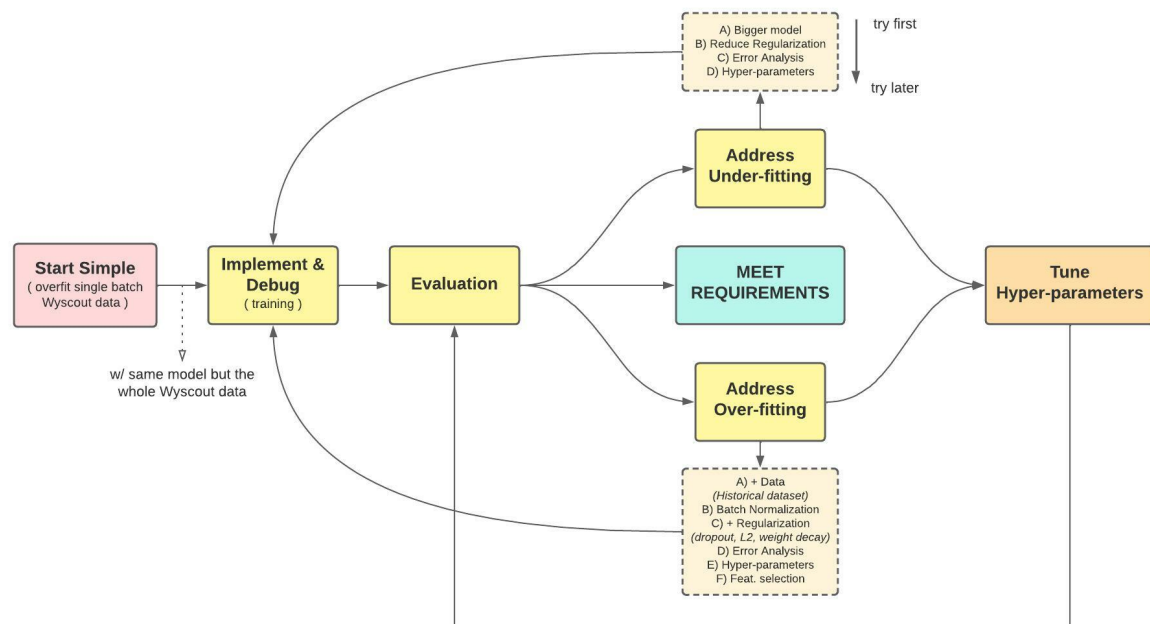


Figure 4.5: Pipeline diagram of the methodology used during the experiments phase in order to state the decision making process when training and evaluating a model.

A continuación se explicarán los experimentos que se han llevado a cabo con modelos MLP simples y configuraciones por defecto. Empezaremos explicando los experimentos para debuguear el correcto funcionamiento de la infraestructura creada para entrenar los modelos.

4.2.1. Artificial Neural Networks experiments

El primer algoritmo de Machine Learning que será utilizado para predecir los resultados serán las Redes Neuronales, en concreto utilizaremos redes MLP. El objetivo de esta parte es encontrar el modelo de red neuronal óptimo para predecir los resultados de partidos de fútbol dadas las características desarrolladas. Durante esta parte, por tanto, realizaremos una serie de experimentos en los que se entrenarán redes neuronales con diferentes

configuraciones (arquitectura e hiperparámetros). Las diferentes configuraciones se irán decidiendo según una heurística basada en el análisis de los experimentos anteriores (Figure 4.5).

La red neuronal diseñada será una MLP que recibirá como input en la primera capa las features de un partidos de , y obtendrá en la salida tres valores, uno por cada clase: victoria local, visitante o empate. Los tres valores de la salida serán normalizados con un softmax con el objetivo de representar la probabilidad de que el sample pertenezca a cada clase. Más concretamente el output del softmax vendría a ser una estimación de la probabilidad de que el target (victoria local, visitante o empate) sea y dado un partido x , es decir una estimación de $p(y|x)$.

Es cierto que el output del softmax de la capa output de una ANN no representa exactamente una probabilidad, principalmente, porque aunque las ANN suelen realizar buenas predicciones no es exacto usar este output como probabilidades verdaderas. En la práctica el output de las ANN no produce probabilidades calibradas, es decir, no representan la probabilidad de que una predicción sea correcta, sino que contienen un sesgo que tienden a predecir probabilidades mayores siendo demasiado confiadas en la clase con mayor probabilidad. Este fenómeno se llama overconfidence [Guo et al., 2017].

Sin embargo, aunque [Guo et al., 2017] presenta una solución para la calibración del output de una ANN, este problema se sale del scope de este trabajo, por lo que asumiremos el output de la función softmax como probabilidades que tienen un margen de error. Además, se hará un análisis del error de estas probabilidades utilizando un intervalo de confianza.

Hemos hablado sobre la interpretación del output de la ANN como probabilidades, sin embargo es esencial hablar sobre las métricas que utiliza el modelo para aprender del error que comete durante el entrenamiento. Las redes que entrenaremos utilizarán Cross Entropy (CE), la función utilizada normalmente para entrenamiento de modelos en problemas de clasificación multiclase. Para un partido n buscamos minimizar la función CE, que tiene este aspecto:

$$L_n = - \sum_{c=1}^C w_{(c)} t_{(n,c)} \log p_{(n,c)} \quad \text{🗨️} \quad (4.1)$$

siendo $p_{(n,c)}$ la función softmax output del partido n para la clase c :

$$p_{(n,c)} = \frac{\exp x_{(n,c)}}{\sum_{j=1}^C \exp x_{(n,j)}} \quad (4.2)$$

Los valores target con los cuales el modelo compara su predicción para medir su error están representados en un vector One-Hot Encoder donde el índice de la clase correcta tiene un 1 mientras que los demás tienen un 0. Nuestro vector One-Hot Encoder tiene una longitud de tres valores, uno para cada clase (victoria local, visitante y empate), siendo empate la primera posición del vector (índice 0), segunda posición es victoria local (índice 1) y la victoria visitante el último puesto (índice 2). Los labels de cada clase de cada partido (es decir cada índice del One-Hot Encoder de cada partido) están representados como $t_{(n,c)}$ en la Eq. 4.1. De esta forma, sólo se penaliza el error en la clase target, ya que los demás labels del One-Hot Encoder están a 0 y desvanecen el error en dichos outputs. Más adelante se discutirá como mejorar la evaluación del error en el entrenamiento por parte del modelo.

Volviendo a la estimación de las probabilidades, estas van integradas en la implementación

de Cross-Entropy que proporciona la librería PyTorch (la cual hemos usado), por lo que en el output del modelo no hemos usado ningún softmax y pasamos directamente el valor que devuelve la capa output a la función Cross-Entropy, que utiliza por cierto un Log-softmax, el cual proporciona ventajas como que es más barato computacionalmente y penaliza más el error; y por otra parte, utilizamos el valor output de la red para calcular con softmax las probabilidades de cada clase y después seleccionar la clase con una máxima probabilidad como predicción. La clase que predécimos finalmente $\hat{t}$ se calcula con la siguiente fórmula:

$$\hat{t}_n = \arg \max_{p(n,c)} \quad (4.3)$$

Finalmente, el accuracy se obtiene calculando la media de acierto, es decir, si $\hat{t} = t$, de todos los partidos. Por otra parte, en un primer momento, el modelo utiliza como optimizador el algoritmo ADAM con una configuración por defecto con un learning rate (lr) de 0.001. Además no se aplica ningún tipo de normalización en las capas de la ANN, y los pesos de las capas se inicialización por defecto. Por último la función de activación utilizada sera ReLU.

Finalmente los datos, procedentes del Event Based Dataset, tienen una dimensionalidad de 20 features. Antes de proceder al entrenamiento los datos serán divididos en dos splits aleatorios, uno de entrenamiento y otro de test, en un ratio inicialmente de 75-25. Además los datos serán normalizadas antes de introducirlos en la ANN. La red entrenará por e epochs, el cual será un valor por optimizar, y además la red será optimizada, es decir sus pesos serán actualizados por cada batch de datos, es decir cada b samples. El tamaño de cada batch (b) es otro hiperparámetro a optimizar, pero en un primer momento este tendrá un valor por defecto de 32 samples per batch. El modelo será entrenado con el set de entrenamiento utilizando 5-folder Cross-Validation para validar el modelo utilizando en cada iteración una parte de los datos para validar. Finalmente el modelo será reentrenado con el conjunto de todo el set de entrenamiento y será validado haciendo inferencia del set de testing. Es importante aclarar que todo el pipeline tendrá asignado una seed para asegurar la reproducibilidad del entrenamiento.

A continuación se detallarán los diferentes experimentos llevados a cabo con esta familia de algoritmos:

Bug-Free Models Para realizar los experimentos se ha tenido que implementar una infraestructura software en la que se ejecute del pipeline de los experimentos (**IMPLEMENTACION**). El pipeline sirve para tener una misma plataforma sobre la cual entrenar todos los modelos (desde los más sencillos a los más complejos), loguear información sobre el entrenamiento y guardar tanto estos logs como los resultados, tal y como se ha explicado en el capítulo de la **Metodología**. Una vez implementada la infraestructura básica y siguiendo la metodología propuesta (Figure 4.5), se han realizado una serie de experimentos para debug esta implementación y asegurarse de que no existen errores ni bugs en el entrenamiento y en el log.

Para debuggear el correcto funcionamiento del pipeline de entrenamiento se ha utilizado un sencillo MLP con una arquitectura de 1 capa de input con 10 neuronas, ninguna capa intermedia oculta y un multi-output de 3 neuronas, una por cada clase a predecir. Con esto, se intenta que la red neuronal prediga la probabilidad de un partido de acabar victoria local, visitante o empate. El input será de 20 dimensiones, una por cada feature de cada registro (cada partido). En total esta red cuenta con 263 parámetros a entrenar. La arquitectura de la red se muestra en la Figure 4.6. Los demás hiperparámetros son los de por defecto, usaremos ADAM como optimizador, Cross-Entropy como loss-function,

ReLU como función de activación y no utilizaremos regularización de ningún tipo.

El experimento de prueba consiste en provocar overfitting de un batch de entrenamiento (8 samples, pero podrían ser incluso menos), y una vez llegados a un error de 0 o muy cercano, comprobar la alineación de las predicciones y los targets, además de mirar los logs que tengan sentido y funcionen correctamente.

Si observamos la curva de aprendizaje, podemos ver, por una parte, como el modelo ha sobreentrenado el batch al máximo, logrando un error de 0 y un accuracy del 100% en el train, mientras que en el test el error ha ido aumentando iteración a iteración. Esto se debe a que el modelo ha aprendido los patrones que siguen los datos del train de memoria, sin tener ninguna capacidad de generalizar cuando se le introducen nuevas muestras.

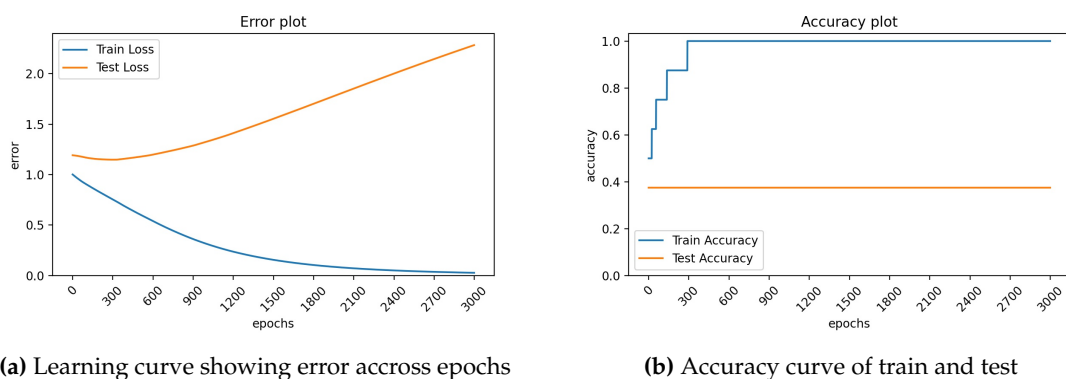


Figure 4.6: Event based data batch overfitted with a 263 parameter MLP.

Otra observación que se puede hacer de esta prueba es que el accuracy obtenido en un experimento puede maximizarse con un error no óptimo. Como se puede apreciar en Figure 4.6 se llega a un accuracy del 100% en el train en el epoch 500, sin embargo, en ese instante el error seguía siendo bastante alto, cercano al 0.8. Esto da señales de que el error del modelo no es el único indicador a fijarse a la hora de analizar los resultados y el rendimiento de un modelo, y que, por tanto, no es necesario obtener el error mínimo para lograr una accuracy buena o incluso óptima. Figure B.1 muestra la evolución de los pesos durante la prueba que se ha realizado para comprobar el correcto funcionamiento de la infraestructura sobre la que se realizaran los experimentos. Se puede apreciar como desde el inicio del entrenamiento la mayoría de parámetros van convergiendo, a diferente ritmo, hacia la distribución de pesos final.

A continuación, se describen los experimentos realizados con la totalidad del dataset y aplicando la metodología propuesta anteriormente. Estos primeros experimentos han sido realizados con el Event Based dataset.

Event Based Dataset Experiments Los primeros experimentos han sido realizados para tener una idea de como se comportan diferentes arquitecturas con estos datos, es decir, que resultados se obtienen con ANN de diferente profundidad y amplitud (neuronas por capa). Estos experimentos se han realizado con la misma configuración que la validación de la infraestructura de experimentos del anterior apartado, es decir con una configuración por defecto. La diferencia es que en este apartado utilizamos todas las muestras del dataset y los batch de entrenamiento que se le pasan a la red es de 32 registros. La red se ha optimizado con ADAM (parámetros por defecto), con un learning rate de 0.001, se

ha utilizado CE como loss-function y ha sido validada con 5-folder CV, tal y como se ha explicado al comienzo de esta sección (4.2.1). Además, los entrenamientos tendrán una duración de 250 epochs, es decir el dataset de entrenamiento será pasado a la red 250 veces. Este valor es un valor inicial que tendrá que ser adecuado según los resultados, pero, a priori, ha sido elegido porque lo consideramos oportuno dado el limitado número de muestras que tenemos.

Los resultados de los experimentos de pruebas de la arquitectura se muestran en la Table 4.2. En esta tabla se puede observar como para redes con más complejidad, es decir, con mayor número de parámetros, el modelo aprende mejor los datos de entrenamiento, eso sí, overfitting considerablemente. Un modelo con 10,000 parámetros llega a obtener un acierto del 90% en el training set, sin embargo, en el test obtiene un acierto de tan sólo el 40%. Los modelos más complejos, que tienen una mayor profundidad y más parámetros son, probablemente, modelos demasiado complejos para los datos que disponemos, tanto en el número de registros (muy limitado) como con la dimensionalidad del input (20 dimensiones). Otro punto interesante, que ya hemos mencionado antes es el hecho de el modelo con el error en el test más bajo no coincide con el modelo con el acierto mayor en el test. Este fenómeno será analizado posteriormente con más detalle. Por otra parte, son los modelos más simples pero que tienen redes más profundas (10-20 capas) los que mejor generalizan los datos de test, sin embargo, este error no se traduce en un buen accuracy y sólo logran un 45% de acierto en el test, frente al 48% que logran los modelos medianos de 2000 y parámetros con sólo 3 capas. Es importante mencionar que cuando hablamos de n capas, no incluimos la capa output pero sí la input, por lo tanto en un modelo de n capas tendríamos $n - 1$ hidden layers además de la capa input y la output.



params. \ layers	Train Error				Test Error				Train Accuracy				Test Accuracy			
	3	4	10	20	3	4	10	20	3	4	10	20	3	4	10	20
500	0.89	0.89	1.07	1.06	1.33	1.32	1.07	1.07	0.59	0.6	0.45	0.45	0.33	0.43	0.43	0.43
750	0.85	0.86	0.86	0.89	1.39	1.39	1.32	1.31	0.62	0.61	0.63	0.67	0.45	0.39	0.3	0.28
1000	0.85	0.87	0.9	0.91	1.37	1.36	1.3	1.31	0.61	0.6	0.64	0.69	0.44	0.31	0.24	0.25
1500	0.83	0.82	0.9	0.9	1.38	1.44	1.29	1.33	0.64	0.63	0.65	0.66	0.32	0.44	0.38	0.35
2000	0.8	0.8	0.89	0.85	1.4	1.44	1.3	1.40	0.65	0.65	0.61	0.59	0.48	0.43	0.43	0.40
3000	0.76	0.74	0.75	0.72	1.5	1.55	1.71	1.88	0.69	0.7	0.66	0.58	0.43	0.4	0.33	0.35
5000	0.67	0.68	0.75	0.7	1.57	1.64	1.82	1.07	0.73	0.72	0.67	0.46	0.48	0.37	0.31	0.42
10000	0.52	0.37	0.7	0.62	1.78	2.49	1.82	2.15	0.82	0.9	0.66	0.74	0.38	0.4	0.42	0.43

Table 4.2: Experiments results of different ANN architectures with default parameters on the Event Based dataset.

En la Figure 4.7 se muestra una comparativa entre cuatro de los modelos mostrados en la Table 4.2. Estos cuatro modelos son detallados en Table 4.3. En la comparativa se aprecia como los modelos más complejos, con más parámetros aprenden más deprisa y además tienen más flexibilidad a la hora de modelar los datos de entrenamiento. Sin embargo, estos modelos más complejos (3000 y 5000 parámetros) su error en el test es considerablemente mayor, lo que significa que estos modelos sobreentrenan excesivamente los datos de entrenamiento. Se aprecia una contradicción, que hemos mencionado anteriormente, en la gráfica del test accuracy en la que se observa como los modelos con más acierto son también los que más error tienen. Por tanto, por una parte, tenemos unos modelos que sufren de underfitting, es decir, la red no es lo suficientemente flexible para modelar los datos de entrenamiento y por tanto en el entrenamiento converge con un error elevado. Esto puede ser debido a diferentes factores no excluyentes:

- **El modelo es demasiado sencillo:** puede ser que el modelo sea demasiado sencillo para la complejidad de los datos. En ese caso se podría enfocar la solución entrenando un modelo con más complejidad, ya sea con más capas o más neuronas en

Modelo	Parameters	Layers	units perm layer
500	443	3	10-15-3
750	743	4	20-10-5-3
3000	3023	4	50-30-10-3
5000	4943	3	70-45-3

Table 4.3

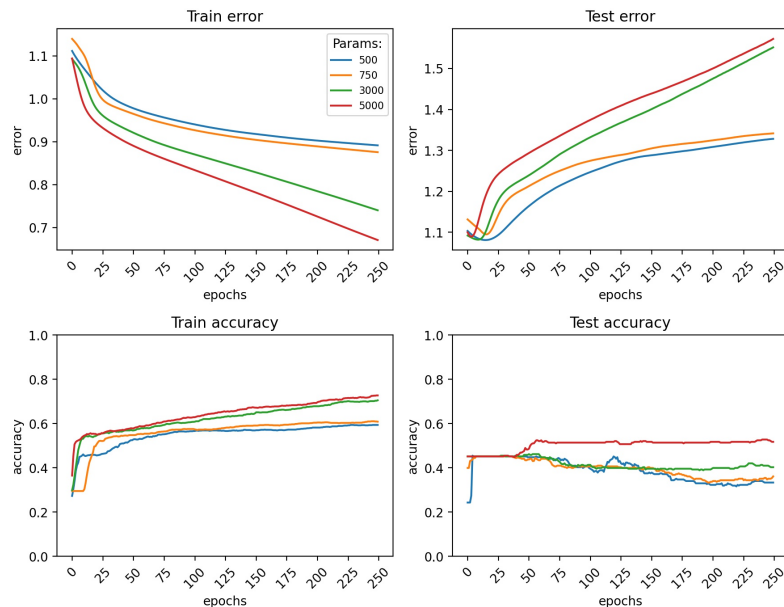


Figure 4.7: Error and accuracy learning curves for the 500, 700, 3000 and 5000 parameter ANNs.

las capas. También se podría reducir la complejidad de los datos reduciendo su dimensionalidad. Esto se hace aplicando alguna técnica de reducción de dimensionalidad.

- **Optimizar hiperparámetros:** estos modelos tienen configuraciones por defecto, por lo que se podrían modificar los hiperparámetros del modelo para mejorar la performance, por ejemplo, se podría aumentar el Learning Rate para que haga "saltos" más grandes cuando con el Descenso por Gradiente actualice los weights o aumentar el número de epochs del entrenamiento.
- **Rapidez del aprendizaje:** el aprendizaje es bastante lento en estos modelos. Se puede optimizar hiperparámetros para mejorar el rendimiento del modelo, pero también se pueden aplicar una serie de métodos para optimizar el rendimiento del modelo y hacerlo más estable y robusto. Algunos de estos métodos son el Batch normalization o la inicialización de los pesos.

Por otra parte, tenemos modelos que sufren mayormente de lo contrario, overfitting, es decir, el modelo aprende bien los datos de entrenamiento pero no sabe generalizar frente a nuevos datos no vistos durante el entrenamiento. Digamos que el modelo memoriza los datos de entrenamiento y acopla al milímetro los parámetros a estos datos. Este es el caso de los modelos de 3000 y 5000 parámetros de la Figure 4.7. En este caso además de optimizar los hiperparámetros como el Learning rate o los epochs, se podría aplicar también otras soluciones:

- **Añadir más datos:** esta es la opción preferible. Es ampliamente aceptado que con una cantidad mayor de datos, y de calidad, es como más eficazmente se reduce el

overfitting. Sin embargo no disponemos de más datos en este dataset, por lo que otras soluciones serán contempladas primero antes de hacer experimentos con un nuevo dataset.

- **Regularization:** sería conveniente aplicar algún metodo para regularizar la ANN y que aprenda a generalizar mejor. Algunas de las técnicas que se pueden aplicar son weight decay, drop out o aplicar pesos a la funcion de error. Los metodos que se utilicen seran explicados.

Las primeras soluciones que se han aplicado han sido con el objetivo de reducir el overfitting y de mejorar el rendimiento de nuestra ANN para que converja en una solución óptima de manera más rápida. Estas soluciones han sido **Batch Normalization (BN)**, **Weight decay** y **Drop-out regularization**, además de cambiar ciertos hiperparámetros de una manera manual e intuitiva para ver que impacto tienen en el modelo.

Para aplicar estos metodos en los experimentos se ha debido modificar y ampliar el código para que soportase las nuevas funcionalidades sin cambiar la manera de entrenar y guardar los resultados y logs. En cambio para los demás cambios no ha hecho hacer ninguna modificación del código, ya que el Weight decay es un parámetro del optimizador de la ANN.

El primer método, Batch Normalization, se usa para poder entrenar ANN de manera más rapida y robusta a través de la normalización (recentrando y reescalando) de los inputs de las capas de la red. Como ya hemos explicado en un capítulo previo, BN corrige la desviación del input de cada capa (Internal Covariate Shift) durante el entrenamiento y los unifica bajo una misma distribución, reduciendo así la interdependencia entre capas. Además, en la capa de BN se entrenan unos extra-parameters para ajustar la distribucion del input y explotar el umbral de non-linearity de la función de activación. Es importante que el input de la función de activación esté en el umbral de non-linearity para reducir la probabilidad de que ocurran problemas de Vanishing and Exploding Gradient y que la ANN converja más rápido en su solución óptima. Es por ello que el BN se suele aplicar al input de la función de activación. Esto permite utilizar un Learning rate más grande ya que ahora la red es más estable. Además, BN reduce el overfitting debido a que al centrar y reescalar los inputs estamos aplicando un ruido, este ruido dificulta a la red a "memorizar" los datos de entrenamiento, especialmente cuando más pequeño sea el tamaño de cada batch. Podemos ver como BN no tiene el mismo impacto en todos los casos. En

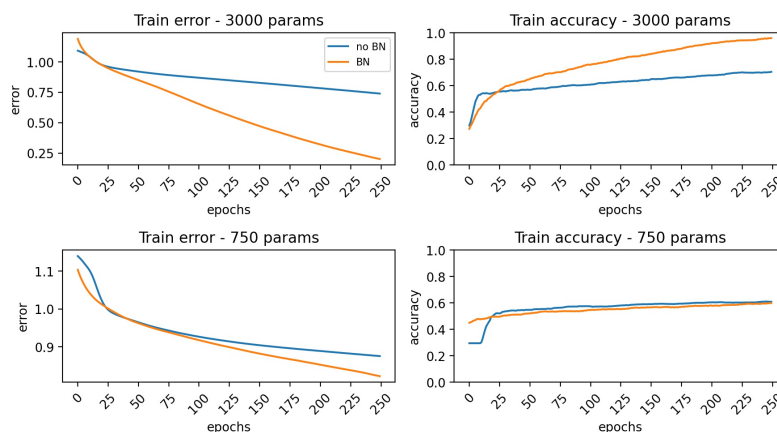


Figure 4.8: Effect of Batch Normalization on training for a 750 and a 3000 parameters model.

redes más complejas, como en esta de 3000 parametros, tiene más efecto y vemos como el

aprendizaje converge más rápido y es más estable. Sin embargo, en redes más simples no tiene el efecto esperado y vemos como el error y el accuracy es más parecido a entrenarlo sin BN.

Sin embargo, BN no ha conseguido reducir el overfitting, de hecho, lo ha aumentado. Para analizar el grado de underfitting y overfitting de un modelo utilizamos el bias-variance decomposition, que como su nombre indica descompone el error del modelo en tres partes, un error irreducible, el bias y la variance, relacionadas estas dos últimos con el underfitting y overfitting respectivamente. El error irreducible sería el error mínimo posible (sea humano o no). En nuestro caso lo consideraremos como 0 ya que es tenemos forma de conocer dicho valor de momento, y por tanto asumiremos que es posible lograr un error de 0. Por otra parte el bias es el training error y la variance es la diferencia entre el error en el test y el train error.

El bias está relacionado con el underfitting debido a que ambos están relacionados con asunciones erróneas del modelo durante el entrenamiento, ignorando éstas relaciones relevantes en los datos. La variance y el overfitting están ambos relacionados con la sensibilidad del modelo frente a variaciones en los datos, teniendo el modelo una sensibilidad excesiva (ya que modelo incluso el ruido) en los datos de entrenamiento que le penaliza frente a nuevos datos en el test. A la hora de aplicar el bias-variance decomposition asumimos que ambos sets de datos (train y test) provienen de la misma distribución [Neal et al., 2018].

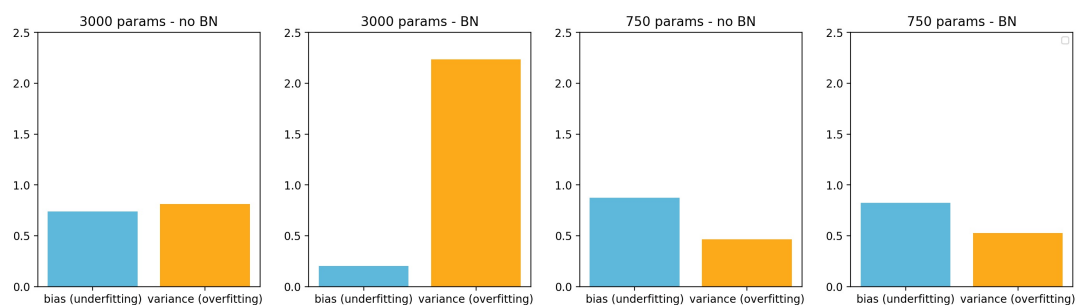


Figure 4.9: Bias-variance decomposition of trained non-BN and BN Neural Networks.

En la Figure 4.9 se muestra que BN reduce el bias (underfitting) sobretodo cuando el modelo tiene la flexibilidad suficiente para modelar los datos (~ 3000 parámetros), mientras que cuando el modelo tiene pocos parámetros (~ 750) tiene poco margen de mejora y de flexibilidad. Por otra parte vemos como BN no reduce en absoluto el overfitting y, de hecho, aumenta por tres veces.

Hemos visto hasta ahora que necesitamos modelos medianamente complejos si queremos tener un modelo con la capacidad de modelar los datos correctamente. Como habíamos mencionado antes, aumentar la complejidad de la ANN ayuda a reducir el error en el training. Sin embargo, ahora debemos también reducir el overfitting para que nuestro modelo tenga un mejor acierto en el test.

Para ello primero hemos realizado varios experimentos con diferentes valores de weight decay. Weight decay es una técnica de regularización en la que aplicamos una penalización L2 a los pesos en la función de error de nuestra ANN con el objetivo de suavizarlos. Normalmente una ANN que está sobreentrenada tiene unos pesos con valores muy grandes (positivos o negativos) que le permiten ser muy flexible a la hora de modelar

todos los datos de entrenamiento. La idea de weight decay es penalizar durante el Back-propagation el hecho de tener pesos con valores muy elevados, obligando a la ANN a buscar una configuración óptima smoothie.

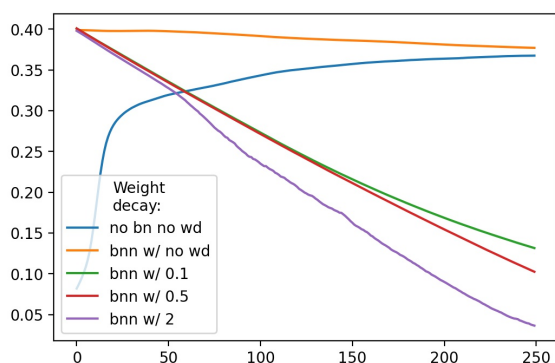
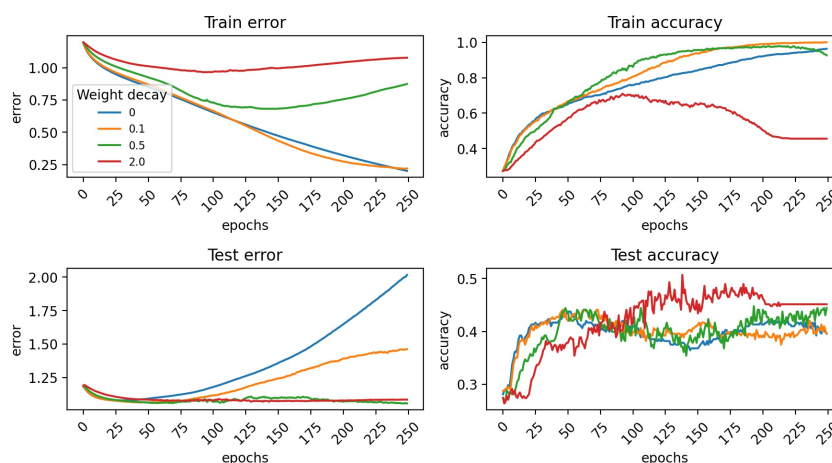


Figure 4.10: Mean of input activation function of a hidden layer during the training for 4 different models: Vanilla ANN without BN or weight decay, ANN with BNN but no weight decay and three ANN with BNN and weight decay of 0.1, 0.5 and 2 respectively.

Figure 4.11: Error and accuracy for train and test experiments on weight decay.



En la Figure 4.10 podemos ver como cuanto más grande es la penalización del weight decay los valores más tienden a 0. Esto es debido a que en estos modelos los pesos tienen hacia 0 debido a la penalización L2 aplicada. Vemos también como el modelo con BN y el que no tiene BN convergen hacia un mismo punto, sin embargo, el de BN ya se inicializa en un valor más cercano por lo que le cuesta menos tiempo converger. Es otra forma de ver la eficiencia de entrenar con BN frente a sin BN.

Sin embargo, la aplicación de weight decay para reducir overfitting no ha dado los resultados deseados (Figure 4.11). Lo cierto es que aunque hace su función de regularizar y ha implicado esto mejores resultados en el test, parece ser que la penalización a la hora de actualizar los pesos es demasiado fuerte y no permite a la ANN modelar bien los datos, especialmente en cuanto al accuracy.

También hemos aplicado otra técnica conocida de regularización llamada Drop-out. Esta técnica consiste en poner a desactivar temporalmente algunos nodos de la ANN (input and hidden layers) creando pues una nueva arquitectura que proviene de la ANN original. Los nodos son desactivados con una probabilidad p . Cuando una ANN busca minimizar su función de error lo hace tomando en cuenta todas las neuronas a la vez, entonces, durante el entrenamiento se crean complejas co-relaciones entre neuronas donde unas "solucionan el error de otras", lo cual ocurre de forma intensa durante el overfitting. La técnica del Drop-out soluciona el problema del overfitting porque al eliminar en cada iteración ciertas neuronas de forma aleatoria estamos creando una nueva arquitectura en

cada iteración (manteniendo los pesos de las neuronas que están activas) y por tanto se fuerza a que las neuronas tengan una interdependencia menor, ya que se actualizan cada vez estando en una arquitectura diferente [Srivastava et al., 2014]. Realmente, Drop-out es una variante de hacer bagging con varios modelos. Es importante mencionar también, que según la literatura relacionada se desaconseja el uso de Drop-out con Batch Normalization [Goodfellow et al., 2016].

Figure 4.12: Error and accuracy for train and test experiments on drop-out.

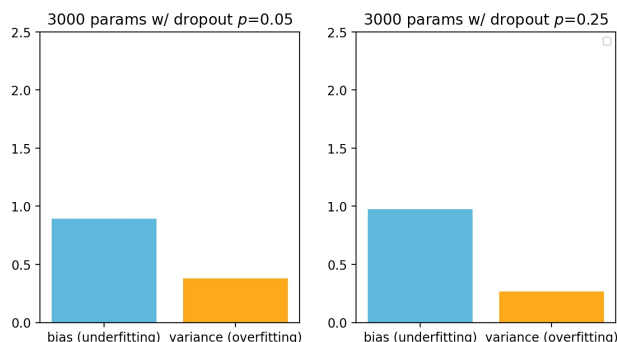
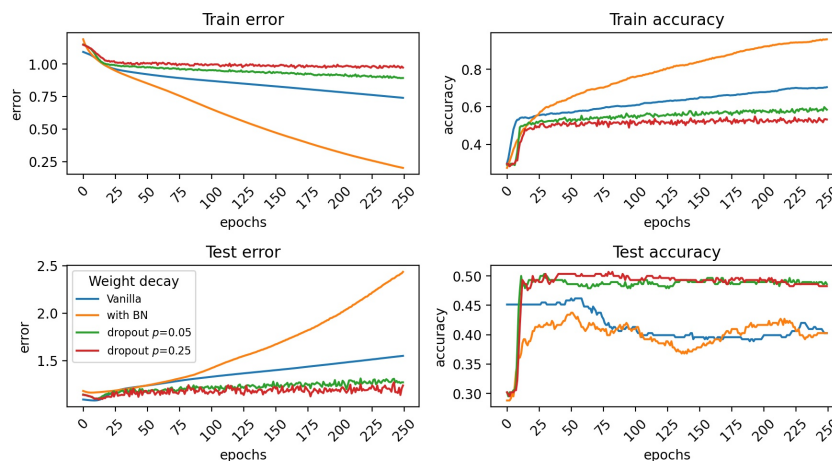


Figure 4.13: Bias-variance decomposition of trained non-BN and BN 3000 parameters Neural Networks.

Hemos realizado varios experimentos con dos valores de p : 0.05 y 0.25. Como se puede apreciar en Figure 4.12, esta técnica también acelera el aprendizaje del modelo, podemos ver como el accuracy se incrementa enormemente en los primeros 25 epochs. Además en este caso vemos mejoras en el accuracy respecto a los experimentos con Weight decay, en los cuales el aprendizaje era irregular y errático, aquí es mucho más regular, rápido y además el modelo logra alcanzar un acierto del 50% en el test. También es importante la disminución del overfitting en los experimentos de Drop-out (Figure 4.13) respecto a los de BN (Figure 4.9).

Hasta el momento se han mostrado varias técnicas para corregir el underfitting y overfitting en el entrenamiento. Sin embargo, estos experimentos explorativos aún pueden ser mejorados ya que han sido realizados independientemente y con muchos hiperparámetros mantenidos por defecto. Estos experimentos han servido para hacernos una idea de que impacto pueden tener estas técnicas en nuestro modelo y como reacciona a añadirle nuevas funcionalidades. Para seguir optimizando nuestro modelo hemos realizado a continuación una búsqueda de los mejores hiperparámetros para nuestro dataset.

Para realizar esta búsqueda de los hiperparámetros óptimos hemos utilizado la técnica de Random Search que consiste en definir un espacio limitado de valores de un hiper-

parámetro y escoger aleatoriamente valores de dicho espacio. Es un buen método para descubrir combinaciones y valores de hiperparámetros poco intuitivos que con Grid Search requeriría más tiempo de búsqueda (Figure 4.14). Con Grid Search tomaría mucho tiempo para cubrir todos los valores y podríamos estar ignorando un valor que minimiza o maximiza la función objetivo. Con Random Search es más probable cubrir más valores del espacio. 🗨️ este primer experimento de Random Search buscamos optimizar

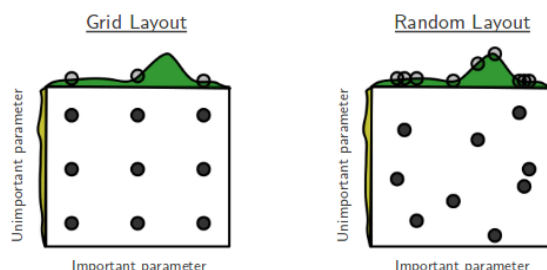


Figure 4.14: Difference between Grid Search and Random Search. The green distribution plot shows how a parameter maximize an objective function.

los hiperparámetros (Figure 4.4) de la ANN utilizando las funcionalidades que hemos mencionado hasta ahora: BN, weight decay y Drop-out. No podemos mostrar, y mucho

Hyperparameter	Search space	Data type	Constraints
Type of net	Vanilla ANN, BN ANN or Drop-out ANN	categorical	
Units per layer	[3-100]	integer	
Layers	2, 3, 4, 9	integer	non-including output layer
Drop-out p	(0, 0.2]	float	only in drop-out
Optimizer	ADAM or Stochastic Gradient Descend (SGD)	categorical	
Momentum	[0-1]	float	only in SGD
Weight decay	[0-1]	float	1/2 choices will be 0
Nesterov	True or False	Boolean	>0 momentum and 0 dampening
Dampening	[0, 0.5]	float	
Learning rate	0.00001, 0.0001, 0.001, 0.01, 0.1	float	
Betas	([0, 1.0], [0, 1.0])	tuple	only in Adam
Scaler	Normalizer, MinMax, MaxAbs, Standarize	categorical	
Batch size	16, 32, 64, 128	categorical	

Table 4.4: Hyperparameters to optimize with Random Search.

menos explicar, cada uno de los experimentos hechos durante el Random Search. Sin embargo, sí podemos hablar de los hiperparámetros (Figure 4.4) que más impactan en el rendimiento del modelo.

En la Figure 4.15 podemos ver los hiperparámetros con mejor accuracy en el test. Vemos que prácticamente todos los experimentos están alrededor de un 45% de acierto en el test. Sin embargo hay unos cuantos experimentos que llegan y pasan el 50% de acierto. Son los modelos con Drop-Out los que mejor acierto logran generalmente, pese a que el Vanilla ANN (dumb model class) también obtiene resultados buenos, muy por detrás están los modelos con BN. De los modelos con Drop-out, los mejores valores para p parecen estar claramente entre 0.1 y 0.2. Por otra parte el número de capas del modelo es bastante importante y se han obtenido mejores resultados con ANN con pocas capas ocultas (2 y 3). Otro hiperparámetro importante es el Learning rate, que aunque todos sus valores se comportan similar respecto al accuracy, es cierto que un LR muy pequeño le dificulta más el aprendizaje a la ANN como vemos en que para un LR de $1e-5$ prácticamente todos los experimentos tienen el mismo accuracy. El weight decay volvemos a ver que no obtiene buenos resultados y salvo con valores prácticamente de 0. El resto de hiperparámetros parece que no condicionan significativamente el acierto del modelo.

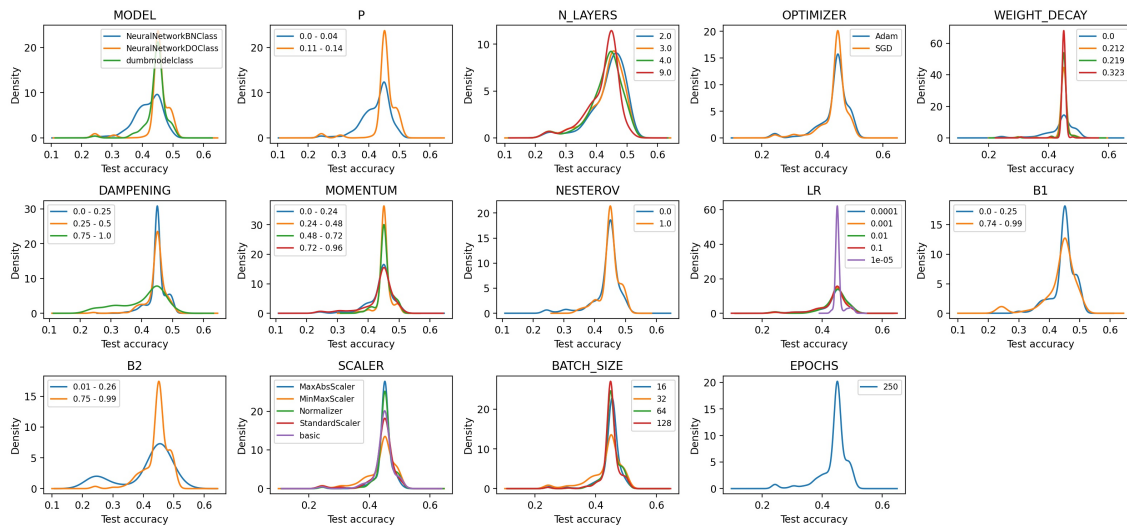


Figure 4.15: Test error distribution for each hyperparameter and value.

Model	Architecture	Optimizer	Lr.	Train err	Train acc	Test. err	Test acc	F1 draws	F1 home	F1 away
Drop-out	73-53-21-3	SGD	0.1	0.947	0.546	1.194	0.512	0.061	0.622	0.487
Drop-out	53-21-3	SGD	0.001	0.989	0.518	1.171	0.510	0.077	0.630	0.444
Drop-out	68-4-7-3	SGD	0.1	0.947	0.546	1.195	0.508	0.031	0.622	0.483
Drop-out	34-11-56	Adam	0.00001	0.992	0.512	1.123	0.508	0	0.619	0.509
Drop-out	42-18-18	Adam	0.001	0.934	0.544	1.201	0.508	0.264	0.621	0.428

Table 4.5: Top 5 best configurations on the Random Search.

Hemos analizado hasta ahora como impactan los hiperparámetros al performance del modelo y qué valores son los óptimos. También hemos aplicado soluciones para reducir el underfitting y el overfitting y tener así el máximo de acierto en el test. Si revisamos cuales los mejores resultados vemos que los mejores resultados tienen poco overfitting, de hecho los 5 mejores se entrenan con Drop-out (Table 4.5). Sin embargo, hay un aspecto que aún no hemos analizado y és, que partidos acierta el modelo y cuáles falla, y cuál es la razón de ello.

F1-score es una métrica muy apropiada para evaluar los resultados de un modelo entrenado con un dataset unbalanced, y como ya hemos mencionado en un capítulo anterior, nuestro dataset no está balanceado (Table 3.9). El F1-score es una métrica que nos indica la relación entre el Recall y Precision de un modelo. La precisión de un modelo indica si lo que has predicho es correcto, es decir el ratio de True Positives entre todos los Positives (True + False). Por otra parte el Recall indica si has predicho bien los samples de una clase, es decir el ratio de True Positives entre todos los samples de dicha clase (True Positives + False Negatives). Finalmente, el F1-score es la media armónica de ambas métricas. En conclusión F1-score indica si el modelo sabe diferenciar correctamente entre las diferentes clases.

Podemos ver en la Table 4.5 que pese a que tienen gran acierto en general (>50%) el F1-score en los empates es practicamente 0 o directamente 0. Vamos a comparar la diferencia entre el modelo con más accuracy y el quinto con más accuracy que es el que tiene también más F1-score. Podemos ver en Figure 4.16 como el primer modelo pese a tener la mejor accuracy general, se le da realmente mal predecir empates. La distribución que siguen las predicciones no se parece en absoluto a la de los valores reales (Table 4.6). Tan sólo el 1% de las predicciones son empates frente al 24% de los valores reales. Vemos que

cuando duda entre victoria local o empate siempre tira hacia la primera, porque no hay ningún caso de victoria local que haya precedido como empate. En el segundo modelo, vemos que mejora considerablemente los empates y aumenta hasta el 11% de las predicciones totales. Sin embargo este modelo sigue lejos del 24% de empates.

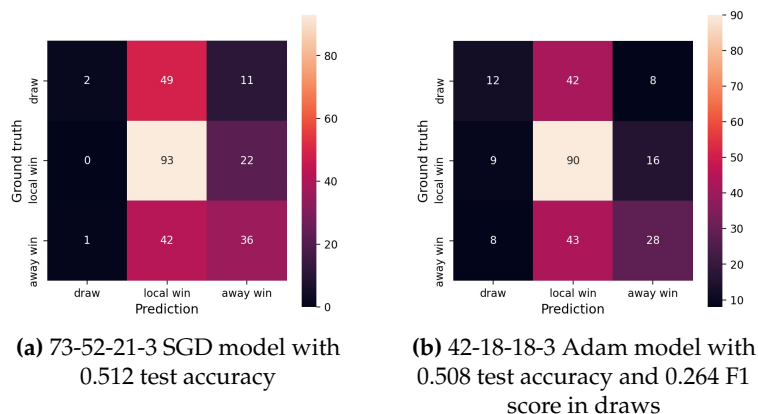


Figure 4.16: Confusion matrix for two ANN with different accuracy in draws.

(%)	Draws	Home wins	Away wins
Ground truth	24	45	31
Predictions model 1	1	72	27
Predictions model 2	11	68	21

Table 4.6: Distribution in percentage (%) of each class in the test set labels and in the test predictions for models 1 (73-52-21-3) and 2 (42-18-18-3).

Vemos que los modelos tienen problemas a la hora de identificar empates (Recall), ya que sólo predice como mucho el 20% de estos. Los modelos infra predicen los empates por lo que hay que buscar soluciones para que el modelo modele más adecuadamente los empates sin penalizar las victorias locales y visitantes.

Para ello, se han propuesto dos soluciones. La primera relacionada con simplificar el entrenamiento para el modelo, y consiste en aplicar a los datos una serie de técnicas de reducción de la dimensionalidad o selección de características. Haremos una serie de experimentos aplicando Variance Threshold y ANOVA para seleccionar las mejores características, PCA para transformar los datos en un espacio dimensional reducido y haremos unos experimentos con una serie de características seleccionadas manualmente tras el análisis de los datos (Table 3.15).

En la Figure 4.17 y en Table 4.7 se muestra como el modelo suele hacer mejores predicciones con inputs de dimensionalidades más grandes ya que le proporciona más información al modelo. Sin embargo, los métodos usados para seleccionar las mejores features no obtienen todos los mismos resultados. PCA obtiene los mejores resultados en todas las diferentes dimensiones a las cuales se ha reducido el input data. Especialmente con 15 dimensiones obtenemos el mejor accuracy de este experimento. Muy cercano está el accuracy obtenido entrenando con las 20 features (todas las disponibles). También mencionar el buen resultado obtenido con las características seleccionadas manualmente tras el análisis de los datos realizado, lo que valida de cierta forma el análisis realizado en un capítulo anterior de la importancia de las variables. Por último mencionar los peores resultados,

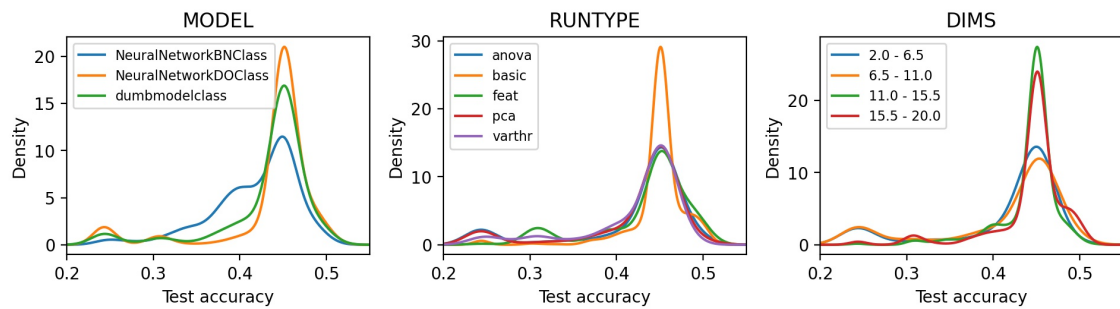


Figure 4.17: Test error distribution for each hyperparameter and value in feature selection experiments.

Method \ Dimensionality	5	10	12	15	20
All features	-	-	-	-	0.512
Manual selection	-	-	0.507	-	-
Variance Threshold	0.457	0.473	-	0.49	-
ANOVA	0.487	0.5	-	0.5	-
PCA	0.512	0.510	-	0.516	-

Table 4.7: Results of feature selection experiments. This table shows the higher accuracy reached for each method employed and new dimensional space of data.

obtenidos con el método de seleccionar los features que superan v umbral de varianza, Variance Threshold.

Además para intentar equilibrar la distribución de predicciones a la de los labels, hemos realizado una serie de experimentos enfocados en encontrar los pesos óptimos que se le asignaron a cada clase en la función de error. En el momento del cálculo del error durante el Forward-propagation se pondera cada clase con su peso para penalizar más unas clases que otras. En este caso se le asigna un peso superior a los empates para que así penalice más el error en los empates y aprenda más del error que comete. Con esto se pretende que el modelo se familiarice más con el empate y sea más sensible a éste. Estos pesos han sido optimizados también con Random Search, optimizando también los demás hiperparámetros (Table 4.4).

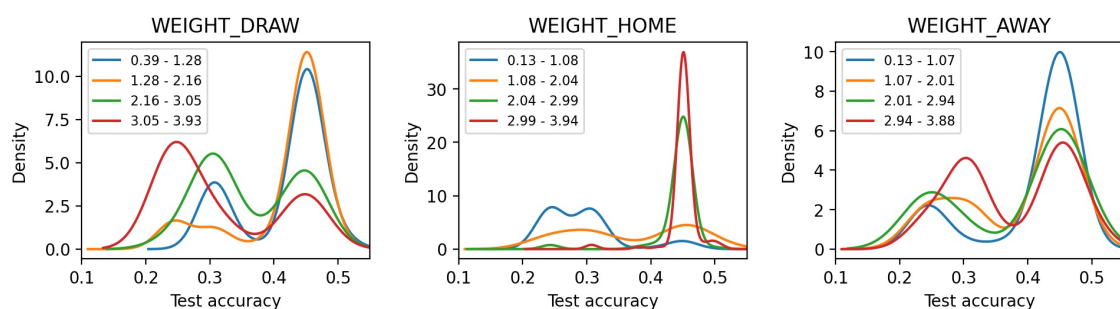


Figure 4.18: Test error distribution for each class loss-weight.

Podemos observar en la Figure 4.18 como estos parámetros tienen un gran impacto en el accuracy del modelo. Sin embargo, la relación entre ellos es compleja porque interactúan entre ellos. Cuando asignamos un peso a una clase, éste no sólo influye en dicha clase sino en el entrenamiento de las demás. Recordemos que los labels que le pasamos

al CE loss (nuestra función de error) son One-Hot Encoded vectors y por tanto en cada partido sólo se penaliza el error en la clase target pese a que la ANN da como output una probabilidad a cada clase. Es decir, si el partido es victoria local, sólo se penalizará el error competido del output de victoria local, sin embargo la probabilidad de empate o victoria visitante no se penalizará y la ANN no aprenderá de ello. Con la configuración por defecto (pesos de 1 para cada clase) todas las clases son penalizadas igual. Sin embargo, al tener un unbalanced dataset, en la práctica, el error en el output de las victorias locales se penalizará en muchas ocasiones, y por lo contrario el output de las visitantes y los empates en muy pocas. Esto provoca que el modelo aprenda más en una dirección que en otras, es decir está sesgado por el gran número de victorias locales. Si además le añades que los empates suelen ser partidos muy igualados y que al haber menos correlaciones de las features con el resultado tiene menos separabilidad en el espacio dimensional de las features, hace que los modelos les cueste mucho aprender y acertar los empates.



Con el objetivo de rebalancear el error se han probado diferentes combinaciones de pesos para cada clase. Vemos en la Figure 4.18 que valores altos para la penalización de los empates provocan una importante caída en el acierto general del modelo ($\sim 25\%$). Esto es debido a que en este caso estamos forzando a realizar grandes cambios en sus pesos cuando comete un fallo en el empate, sesgando el aprendizaje del modelo hacia una sobre predicción de empates. En la Figure 4.19 se muestra la diferencia en los gradientes con diferentes valores de loss-weights. Por tanto, el valor de estos hiperparámetros también afecta el rendimiento del modelo en cuanto a la robustez y estabilidad del aprendizaje. Cuando hablamos de que hay fuertes interacciones entre los tres hiperparámetros es que no sólo importa el valor absoluto del error weight sino el ratio o diferencia entre los tres (Figure 4.20). Si aplicamos tres ratios pequeños pero que una clase tiene la mitad de penalización que los otros, este será penalizado la mitad que los demás y por tanto el modelo potencialmente predecirá menos partidos de dicha clase.

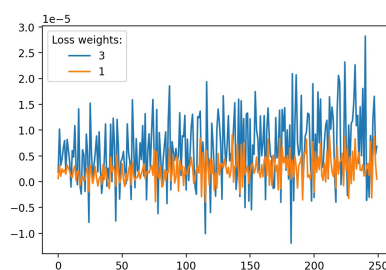


Figure 4.19: Input layer gradients during training with loss-weights of 1 and 3 for all the classes.

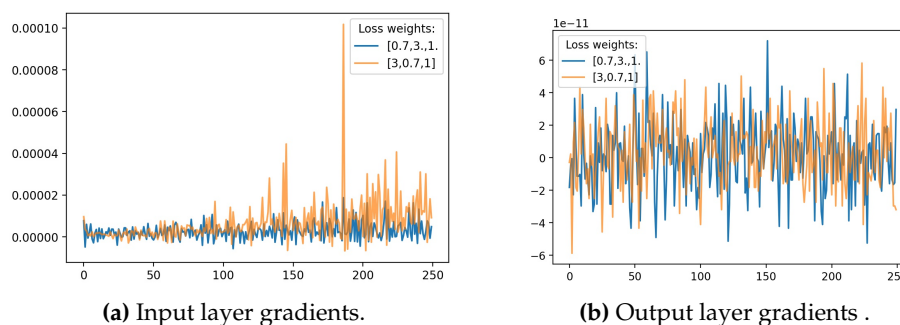


Figure 4.20: Loss-weights focusing in home wins (blue) and draws (orange).

Finalmente, los pesos que mejor resultado a la hora de mejorar la distribución de la

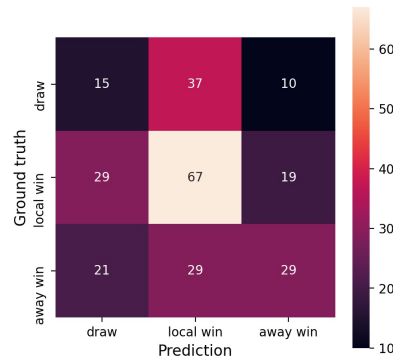


Figure 4.21: Confusion matrix of a model with the following loss weights: draw: 1.2, home win: 0.8 away win: 1.0.

(%)	Draws	Home wins	Away wins
Ground truth	24	45	31
Loss weights: 1.2 - 0.8 - 1.0	25	52	23
Loss weights: 1.1 - 0.9 - 1.0	6	54	40
Loss weights: 1.2 - 0.7 - 1.1	24	34	42

Table 4.8: Distribution in percentage (%) of each class in the test set for different loss weight values.

predicción han dado han sido los que están alrededor de 1. Utilizando un peso ligeramente mayor en los empates que en las victorias locales y visitantes se consigue que el modelo mejore también en la predicción de empates (Table 4.8), pero eso sí, reduciendo ligeramente el acierto total del modelo (Figure 4.21).

Hasta ahora hemos explicado soluciones para reducir el overfitting y underfitting, soluciones para mejorar el rendimiento del modelo para que aprenda más rápido y de manera más estable en el entrenamiento y hemos aplicado también soluciones para que el modelo haga predicciones de más calidad y variadas. Hemos mejorado los resultados desde los primeros experimentos donde los modelos tenían un acierto del 48% mientras que ahora hemos llegado a tener un acierto del 51.6%. Además hemos mejorado el acierto de los empates, donde los primeros modelos tenían un F1-score de aproximadamente 0, es decir casi ignoraba los empates. Ahora hemos logrado un F1-score en los empates de 0.26 y hemos logrado ajustar la distribución de predicciones a la distribución de los labels. Sin embargo, creemos que podemos obtener aún mejores resultados, tanto de accuracy como de separabilidad de las clases.

Dadas las limitaciones que ofrece este dataset de número de registros para entrenar y testear, hemos creado un nuevo dataset (3.2.2) con nuevas features y que tiene 10 veces más registros que el utilizado hasta ahora. Con más datos y, por tanto variabilidad, esperamos que el modelo pueda aprender nuevos patrones y relaciones en los datos.

Historical Dataset Experiments El propósito de los siguientes experimentos es mejorar los resultados de nuestros anteriores experimentos en base a entrenar nuestro modelo con un dataset más grande y con otro tipo de información. Como hemos mencionado en un capítulo anterior, el Historical Dataset contiene datos acerca del estado de forma a corto y largo plazo de los dos equipos que juegan un partido. Con esto, se busca que el modelo tenga no sólo información de los últimos n partidos sino también de meses y años atrás. En el anterior modelo podíamos tener el caso de un equipo, como el Chelsea

o el Bayern Munchen, que viniese de una mala racha y que el modelo automáticamente le diera como ganador al equipo rival ya que sólo se fija en los últimos n partidos. Ahora el modelo tomará también en consideración los últimos meses y años de ambos equipos.

Como también hemos mencionado anteriormente, tenemos varias versiones de este dataset, las cuales compararemos. Para ello, como en los experimentos con el Event-Based dataset, realizaremos una Random Search para encontrar los hiperparámetros óptimos de los modelos entrenados con estos datasets. Los hiperparámetros a optimizar son los mismos que en los experimentos anteriores (Figure 4.4). Además también se optimizaran los pesos de cada clase en el cálculo del error del modelo. Ahora, en cambio, los entrenamientos duraran 100 epochs en vez de 250 como con el anterior dataset. Esto se debe a que ahora el dataset es mayor y seguramente no haga falta iterar sobre él tantas veces, además de por que así evitamos que el tiempo de ejecución sea sumamente largo.

Method	Historical Complete				Hist.Longterm Basic				Hist. Longterm Complete			
	3	8	15	23	5	15	25	55	10	25	50	140
All features	-	-	-	0.495	-	-	-	0.521	-	-	-	0.545
ANOVA	0.488	0.486	0.492	-	0.502	0.511	0.516	-	0.512	0.526	0.526	-
PCA	0.491	0.49	0.495	-	0.501	0.503	0.512	-	0.512	0.514	0.515	-

Table 4.9: Comparison of the accuracy of the Random Search experiments over the three experiments. This table shows the higher accuracy reached for each method employed with each new dimensional space of data and each Dataset.

Como podemos ver en la Table 4.9 los datasets con información a largo plazo tienen un acierto significativamente mayor. De hecho, los dos datasets Longterm consiguen un acierto que no habíamos conseguido hasta el momento. Con el dataset Longterm Complete (accuracy de 54.5%) incluso se iguala el Baseline de las cuotas deportivas (Figure 4.1). En la gráfica izquierda de la Figure 4.22 se muestra la matriz de confusión del modelo con 54.5% de accuracy. En la figura anterior también mostramos la matriz de con-

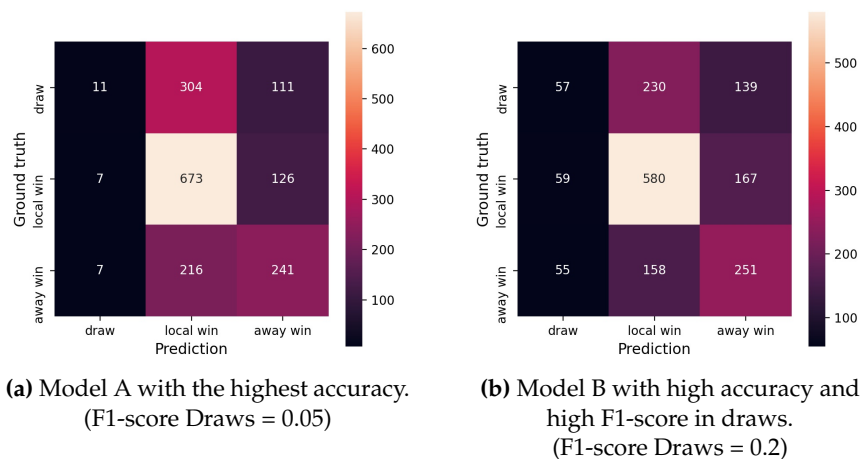


Figure 4.22: Confusion matrix for two models trained with Historical Longterm Complete dataset.

fusión de otro modelo entrenado con los mismos datos (Historical Longterm Complete) pero que logra acertar más empates. En el gráfico del AUC-ROC (Figure 4.23) podemos ver mejor la diferencia entre los dos modelos a la hora de diferenciar entre clases. Vemos que la diferencia es menor de lo que se observa en la matriz de confusión o en el F1-Score, de hecho el índice AUC es prácticamente igual en las tres clases. Esto es debido a que el AUC-ROC se calcula con las probabilidades que saca el modelo.

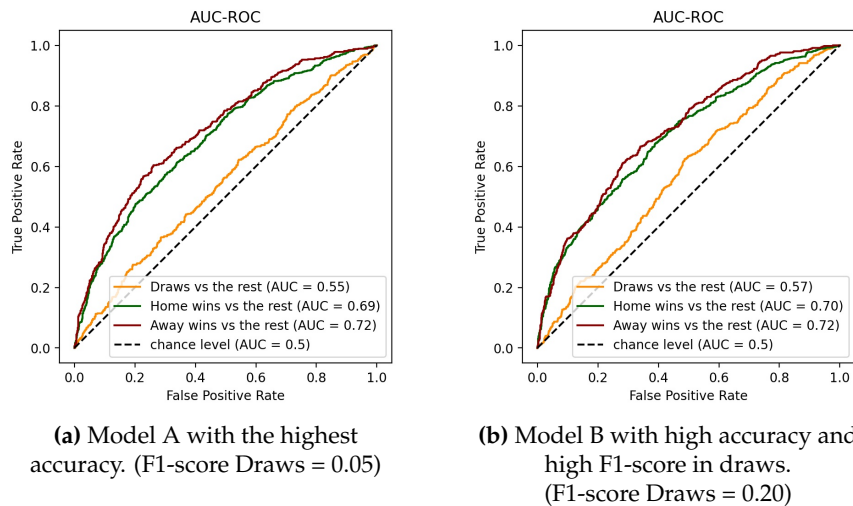


Figure 4.23: AUC-ROC for two models trained with Historical Longterm Complete dataset.

Div.	Season	Match	Res.	Model A				Model B			
				Proposal	pred D	pred H	pred A	Proposal	pred D	pred H	pred A
FRA	2019-20	Reims - Nice	0-0	D	0.35	0.34	0.31	A	0.33	0.25	0.42
GER	2008-09	M'gladbach - Wolfsburg	0-0	H	0.34	0.37	0.29	D	0.36	0.3	0.34
GER	2014-15	Frankfurt - Hoffenheim	3-0	H	0.34	0.35	0.3	D	0.36	0.3	0.34
ITA	2018-19	Juventus - Sampdoria	1-1	H	0.23	0.6	0.17	H	0.24	0.62	0.14
ITA	2020-21	Parma - Fiorentina	0-0	H	0.28	0.41	0.31	D	0.36	0.32	0.32
SPA	2011-12	Villareal - Barcelona	0-0	A	0.24	0.22	0.54	A	0.33	0.24	0.43
SPA	2007-08	Real Madrid - Levante	3-0	H	0.25	0.61	0.14	H	0.24	0.62	0.14

Table 4.10: Examples of outputs of model a) and b) with each actual match outcome, each model proposal and probabilities for each class.

En la Table 4.10 se muestran varios ejemplos de la diferencia de predicciones entre el modelo con más accuracy (Modelo A) y el que tiene F1-score Draw = 0.2 (Modelo B). Como vemos las predicciones no son muy diferentes salvo en ciertos partidos como el Parma vs Fiorentina. Esta es la razón por la que la curva ROC de ambos modelos no es tan diferente. Sin embargo F1-score muestra una diferencia mayor entre ambos modelos porque se basa en el accuracy. El accuracy se calcula de forma booleana (se acierta o se falla, 1 o 0) por lo que la información intermedia (los grises intermedios) que muestran las probabilidades se pierden. Como vemos en el M'gladbach vs Wolfsburg en el segundo modelo se acierta por muy poco, en cambio en el primer modelo se falla por muy poco, siendo el error cometido insignificante, sin embargo el accuracy te lo penaliza de la misma manera. Como sabemos que las probabilidades que da una ANN no son totalmente precisas (como probabilidad), podríamos definir un Intervalo de Confianza que marque el margen de error nuestras predicciones. De esta forma veríamos si dentro del Intervalo de Confianza el modelo acertaría el resultado del partido o no.

Match	Res.	15 _H	15 _A	60 _H	60 _A	1825 _H	1825 _A	Historical Complete				Historical Longterm Complete			
								Proposal	D	H	A	Proposal	D	H	A
Everton - Man Utd	1-2	0	0.5	1.86	1.16	1.39	1.79	H	0.21	0.62	0.17	A	0.33	0.24	0.43
Elche - Real Madrid	0-0	3	1.5	1.86	2.25	1.03	2.43	A	0.26	0.36	0.37	A	0.34	0.23	0.43
Bremen - Hertha	2-0	3	1.5	2.5	1.12	1.17	1.32	H	0.14	0.77	0.09	H	0.34	0.42	0.23
Valladolid - Barça	0-2	2	2.3	1.37	1.55	1.08	2.28	H	0.27	0.38	0.35	A	0.34	0.21	0.45

Table 4.11: Examples of how lags influence output probabilities. 15_x, 60_x and 1825_x are the points ganined for the last n days for each team, and D , H and A are the probabilities given in the output by each model.

Por otra parte, añadir las features de longterm han proveído al modelo de una valiosa información que hasta ahora no tenía, que és, la historia y el status de un club. Con los lags de los partidos del últimos año o incluso varios años le damos al modelo información del éxito que ha tenido un equipo a largo plazo. Ahora el modelo deberá aprender como relacionar las variables de corto plazo (varias semanas o meses) con las de largo plazo (muchos meses o años). Si nos remitimos al acierto de los modelos con ambos tipos de datos vemos que sin longterm el modelo logra un acierto del 49.5% mientras que con variables longterm llegamos al 54.5%.

En la Table 4.11 se muestran algunos ejemplos de como cambian las predicciones de un modelo entrenado sólo con variables a corto plazo (Historical Complete) y otro que utiliza las dos tipos de variables (Historical Longterm Complete). Hasta ahora los modelos solo recibían como input las variables 15_x y 60_x y se basaban en ellas para predecir. Vemos que, de esta manera, partidos como un Valladolid vs Barcelona lo daban como ganador al equipo local por tener ambos equipos una racha reciente similar, y probablemente, le da la victoria al Valladolid por el factor local. En cambio, vemos que añadiéndole información de la racha a largo plazo el modelo sabe que si está igualado en el corto plazo, el equipo históricamente mejor tiene más probabilidad de ganar. Aún así, vemos que el modelo le da un 33% de probabilidad al empate, algo totalmente lógico. Vemos en otros partidos como el Elche vs Real Madrid o Werden Bremen vs Hertha que aunque no cambie el pronóstico final, las probabilidades son más coherentes con el lag de 1825_x .

Hemos mencionado ya anteriormente que nuestro modelo sólo penaliza el error cometido en la clase target (es decir si el partido es una victoria visitante, sólo se penaliza el output de la victoria visitante). Lo que no hemos mencionado aún es que, además, se penaliza de igual forma todos los resultados independientemente de la diferencia de goles, es decir, se penaliza igual equivocarse en un 2-1 que en un 5-0. Para obligar al modelo a aprender más eficientemente, y penalizar sus errores má graves, hemos propuesto una modificación del label que se le pasa a la función de error.

El nuevo label seria el resultado de multiplicar el One-Hot Encoded vector del label original por la diferencia de goles del partido. En caso de empate (la diferencia es 0) se deja el label original. Es cierto que, de esta manera, las victorias locales y visitantes se penalizaran (cuando haya más de un gol de diferencia) más que los empates, lo cual perjudicará las predicciones de los empates. Por eso, se ha propuesto también escalar con una función MinMax entre 0 y 1 las predicciones de cada clase. De esta manera los empates tendrán siempre una penalización de 1, mientras que las victorias locales y visitantes estaran escaladas (entre 0 y 1) en función de la diferencia de goles.

Por último, como hay muchos partidos con poca diferencia de goles, lo que podria hacer que el error fuera generalmente muy bajo y el modelo aprendiese despacio, se ha decidido aplicar un factor multiplicador f . Este factor multiplica todas las columnas escaladas para aumentar el error en un factor de f . Este hiperparámetro f se deberá optimizar como cualquier otro hiperparámetro. Siendo n el numero de partidos del training set, c la clase target, siendo tres el número de clases, t la matriz ($n \times 3$) con los One-Hot Encoded original de los labels del training set, t' la nueva matriz de los labels de cada partido ($n \times 3$), d el vector de diferencia de goles en los partidos del training set ($n \times 1$) y f el factor multiplicador (escalar), la transformación aplicada es la siguiente:

$$t'_c = \minmax_c(d \cdot t) \cdot f \quad (4.4)$$

En la Table 4.12 podemos ver los resultados de la optimización con Random Search de

Factor	Test metrics	Hist.Longterm Basic			Hist. Longterm Complete		
		error	accuracy	f1-score _{draws}	error	accuracy	f1-score _{draws}
no factor f (original)		0.98	0.521	0.2	0.98	0.545	0.15
label · goal diff. (no f)		1.9	0.52	0.02	1.88	0.523	0.02
$f = 2$		0.17	0.519	0.3	0.14	0.514	0.29
$f = 5$		0.34	0.516	0.31	0.27	0.516	0.29
$f = 10$		0.67	0.519	0.3	0.55	0.52	0.31

Table 4.12: Results of factor experiments. It shows the model with the minimum test error and maximum test accuracy and also the mean of the f1-score of each factor f . The first two experiments are the original label format and the label multiplied by the goal difference of the match (with no scaling or factor).

modelos con diferentes factors f . Vemos que el factor impacta considerablemente en la predicción de empates. Aplicando un factor de 2,5 o 10 la media de f1-score de los empates aumenta un 50% respecto a la forma original de los labels. Como habíamos pronosticado antes, si no escalamos los labels una vez multiplicados por la diferencia de goles, tendremos labels muy grandes para los partidos que acaben en goleada, especialmente en comparación con los labels de los empates (que son 1 siempre), y por tanto la función de error dará un error enorme para dichos partidos sesgando el aprendizaje y perjudicando los pronósticos de los empates (que en comparación no se aprenderá nada). Como vemos, esto se ha hecho realidad y el f1-score para los empates es tan sólo de 0.02, además el error es el más alto de todos los experimentos (~ 1.9).

Es interesante ver como los resultados son muy similares con los dos datasets, lo que tiene sentido ya que esta modificación afecta solamente a los labels y el cálculo del error y no a las features. El acierto en cambio sí que cambia más significativamente, en el dataset Basic el acierto disminuye muy poco de la versión original a la de los factores, sin embargo, el dataset Complete vemos que hay un descenso de hasta el 3% en el acierto. Pese a esto, el descenso del accuracy se compensa con una predicciones más balanceadas y repartidas.

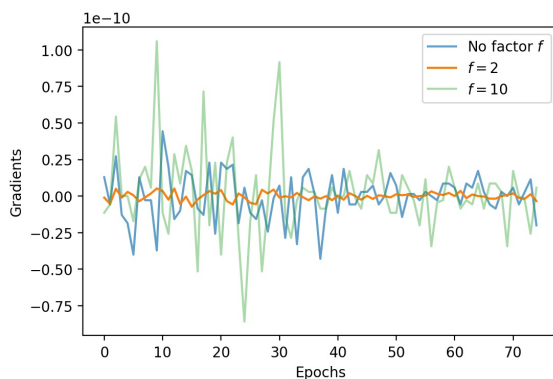


Figure 4.24: Gradients during ANN training for 3 different values of factor f .

Como hemos explicado, el valor del factor f condiciona directamente el valor del error. Los dos valores son directamente proporcionales, para f grandes habrá un error mayor que para f pequeñas. La magnitud del error, a su vez, repercute en la actualización de los pesos de la ANN durante el Back-propagation. En la Figure 4.24 se muestran los gradientes de una Hidden layer para distintos valores de f . Como se puede observar, escoger un valor de f demasiado pequeño puede provocar que el error sea muy pequeño y por tanto los gradientes inapreciables. Este problema se propaga a lo largo de la ANN a las capas anteriores y se le conoce como Vanishing Gradient problem. La consecuencia de esto es un modelo que aprenderá mucho más lentamente ya que los gradientes que le

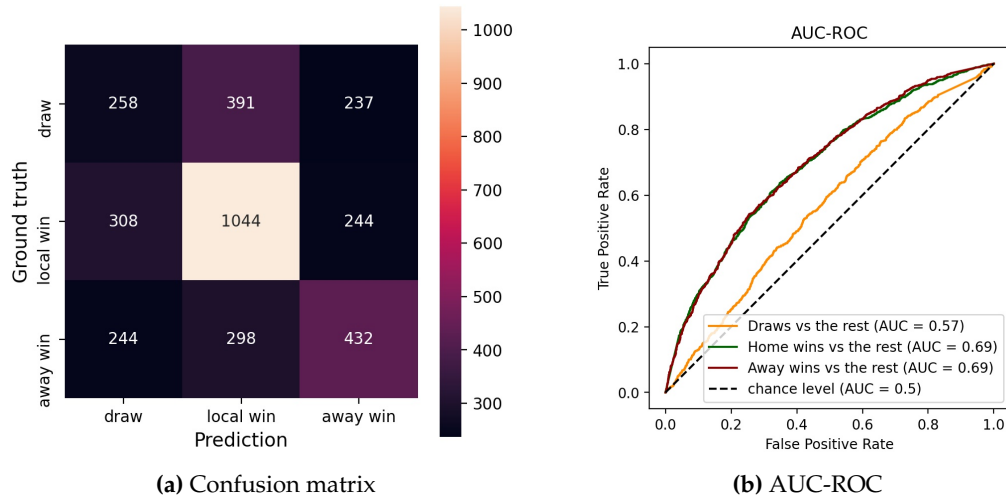


Figure 4.25: Best model with $f = 10$ with accuracy of 0.519 trained with Historical Longterm Complete dataset.

llegarán a las neuronas a la hora de actualizarse serán prácticamente 0.

En la Figure 4.25 se muestra el modelo con un mejor F1-score en los empates (0.3). Es un modelo entrenado con el Historical Longterm Complete dataset, con un factor $f = 10$. La ANN tiene 3000 parámetros en 4 capas, se ha entrenado con Adam como optimizador y regularizado con Drop-out con $p = 0.07$, además ha utilizado pesos en la loss-function (draws = 1.1, home = 0.9 y away = 1.0), y el entrenamiento ha sido de 100 epochs con un Batch size de 32 samples. Han sido utilizadas todas las features para el entrenamiento (140).

Si analizamos algunos ejemplos de las predicciones en el test de nuestro modelo original y de un modelo entrenado con $f = 10$ (Table 4.13) vemos que las diferencias son significativas. Como hemos dicho aumentar el factor implica aumentar el error cometido por el modelo y por tanto forzar al modelo a realizar cambios más grandes para corregir dichos errores. En estos ejemplos vemos como con un factor elevado las decisiones del modelo son más contundentes, hay más diferencia entre las diferentes probabilidades que saca en el output, es decir está más seguro del pronóstico que hace. A su vez, al penalizar más en los empates, vemos que balancea mejor las predicciones, siguiendo estas una distribución más parecida a la de los labels.

Match	Res.	15 _H	15 _A	60 _H	60 _A	1825 _H	1825 _A	NO Factor (original)				Factor $f = 10$			
								Proposal	D	H	A	Proposal	D	H	A
Blackburn - Arsenal	1-0	1.5	0.5	2	1.33	1.43	2.02	H	0.31	0.36	0.33	D	0.37	0.3	0.33
Racing - Zaragoza	0-0	0.5	2	0.88	1.5	1.26	1.24	H	0.30	0.36	0.34	D	0.38	0.3	0.32
Milan - Roma	0-1	0	3	1.33	1.88	2.15	1.87	H	0.28	0.54	0.19	D	0.367	0.368	0.265
Betis - Real Madrid	0-3	0.5	3	0.33	2.67	1.18	2.4	A	0.25	0.12	0.63	A	0.24	0.04	0.72

Table 4.13: Examples of how lags influence output probabilities. 15_x, 60_x and 1825_x are the points ganined for the last n days for each team, and D , H and A are the probabilities given in the output by each model.

En la Table 4.14 vemos realmente el impacto que han tenido los factores según la diferencia de goles del partido. Vemos que no hay una diferencia significativa enntre los diferentes valores de f . En los empates dobla el acierto del modelo original (sin factor), aunque esta mejora en los empates se penaliza especialmente en los partidos con 1, 2 y 4

Goal difference	NO Factor	Only goal difference	$f = 1$	$f = 5$	$f = 10$
0	0.13	0	0.29	0.28	0.29
1	0.57	0.64	0.49	0.49	0.49
2	0.68	0.7	0.58	0.59	0.59
3	0.77	0.81	0.72	0.72	0.72
4	0.86	0.84	0.75	0.74	0.73
5	0.95	0.98	0.98	0.96	0.98
6	1	1	1	1	1
7	1	1	1	1	1
8	1	1	1	1	1

Table 4.14: Test accuracy comparison between a model with no factor and high f1-score for draws (0.2) and models with different f values. All the models have been trained with the same hyperparameters and data.

goles de diferencia donde realiza un 10% menos de aciertos. También es interesante ver como en todas las variantes se acierta practicamente todos los resultados de las goleadas por 5 o más goles de diferencia.

Llegados a este punto, hemos finalizado con los experimentos basados en ANN. Estos han sido los experimentos principales de este trabajo, sin embargo, creemos que es interesante realizar experimentos con otro algoritmo de Machine Learning para comprobar que estamos obteniendo los mejores resultados posibles con ANN. Por eso, ahora presentaremos una serie de experimentos realizados con el algoritmo LightGBM.

4.2.2. LightGBM Based Experiments

LightGBM es un tree-based algorithm de ML open-source basado en Gradient boosting que es ampliamente utilizado actualmente para problemas con gran cantidad de datos. LightGBM es cada vez más utilizado por delante de otros algoritmos de Gradient Boosting por su rapidez y resultados en problemas de clasificación y regresión. De hecho, ha conseguido recientemente excelentes resultados en datos tabulares, por encima incluso de ANN y otros algoritmos de ML más clásicos. Por estas razones y por que nos resultaba más familiar hemos escogido este algoritmo para nuestros experimentos.

Este algoritmo basado en árboles obtiene resultados state-of-the-art en datos tabulares por ser capaz de manejar datos numéricos y categóricos a la vez. En los modelos de aprendizaje automático tradicionales, las variables categóricas se convierten típicamente en variables numéricas a través de técnicas como la codificación one-hot. Esto puede llevar a espacios de características de alta dimensionalidad y dispersos, lo que puede tener un impacto negativo en el rendimiento del modelo y ralentizar los tiempos de entrenamiento. Sin embargo, LightGBM utiliza un algoritmo único que puede manejar directamente las características categóricas mediante la creación de una división basada en histogramas en lugar de utilizar la codificación one-hot. Esto da como resultado modelos más eficientes y precisos que pueden manejar espacios de características de alta dimensionalidad y dispersos con facilidad.

La optimización de hiperparámetros la realizamos utilizando el framework Optuna. Este framework optimiza creando unas combinaciones de hiperparámetros, las cuales, se evalúan mediante una función objetivo que mide el rendimiento del modelo. Optuna utiliza esta evaluación para ajustar la estrategia de muestreo y enfocarse en las combinaciones de hiperparámetros que parecen más prometedoras. A medida que se realizan más pruebas, Optuna  refinando su búsqueda hasta encontrar la combinación de hiperparámetros que maximice el rendimiento del modelo. La estrategia de búsqueda que usa se llama

Bayesian search. 

En la búsqueda bayesiana, se utiliza una distribución de probabilidad para modelar el espacio de búsqueda de los hiperparámetros. A medida que se realizan más pruebas, la distribución de probabilidad se actualiza con los resultados de las pruebas anteriores, lo que permite que la búsqueda se enfoque en las áreas más prometedoras del espacio de búsqueda.

La estrategia de búsqueda bayesiana de Optuna se basa en el algoritmo TPE (Tree-structured Parzen Estimator), que es un método eficiente y escalable para la optimización de hiperparámetros. TPE utiliza dos modelos de probabilidad para modelar las regiones prometedoras y no prometedoras del espacio de búsqueda de los hiperparámetros, y se enfoca en la región más prometedora para seleccionar la próxima combinación de hiperparámetros a probar.

En estos experimentos, análogamente a los experimentos con ANN, buscamos minimizar el error en el test. Además los experimentos han sido realizados con los dos datasets que mejores resultados han dado con ANNs (Historical Longterm Basic y Historical Longterm Complete), y realizaremos experimentos tanto con el dataset completo como aplicando ANOVA y PCA.

Model \ Test metrics	Hist.Longterm Basic		Hist. Longterm Complete	
	error	accuracy	error	accuracy
Basic LightGBM	0.99	0.527	0.99	0.525
ANOVA 25 dims	0.997	0.524	0.989	0.525
ANOVA 50 dims	0.992	0.532	0.987	0.529
PCA 25 dims	1.02	0.506	0.1	0.513
PCA 50 dims	1.01	0.512	0.995	0.519

Table 4.15: LightGBM experiments resume.

En la Table 4.15 se muestran los resultados de los experimentos de LightGBM. Como vemos LightGBM alcanza un acierto realmente bueno con ambos datasets, pero contrariamente a las ANN, tiene mejores resultados con el dataset Historical Longterm Basic. Además parece ser que seleccionando las mejores características según ANOVA mejoramos el rendimiento del modelo respecto a utilizar el dataset completo. Sin embargo los modelos de LightGBM no consiguen modelar bien los empates, a diferencia de las ANNs (Figure 4.26), o al menos no por sí solas (sin añadirle ninguna funcionalidad extra). De hecho, el AUC score de los LightGBM es el más bajo de los que hemos obtenido hasta ahora.

Una característica que tiene LightGBM muy interesante es que es más explicativo con sus decisiones (predicciones) que las ANNs. De hecho, la propia librería de LightGBM que usamos ofrece un método que te muestra las variables más importantes e influyentes para el modelo a la hora de realizar las predicciones. Vemos en la Figure 4.27 que las variables más importantes, como lo sospechábamos, son las variables a largo plazo, especialmente las de la media de puntos obtenidos en los últimos 6 meses, 1 año y 5 años. También es importante mencionar como las variables del equipo local influyen más en las predicciones del LightGBM, seguramente debido a la distribución de los resultados a favor de las victorias locales. Las variables a corto plazo tienen considerablemente menos importancia para el modelo, colándose tan sólo los goles del equipo visitante en los últimos 2 meses entre las 20 variables más importantes. Por último, las variables de goles

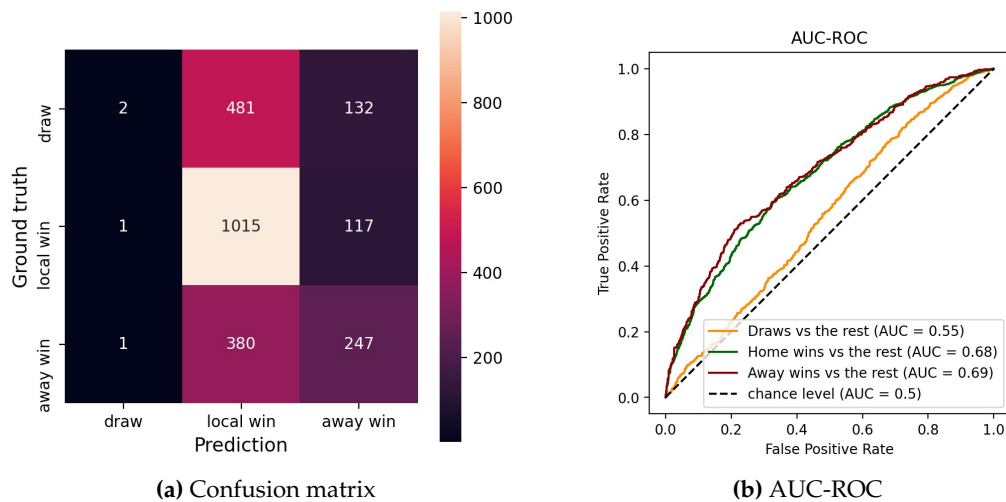


Figure 4.26: LightGBM model with highest test accuracy.

y puntos obtenidos por el equipo local en los últimos 10 partidos entre ambos equipos (derby features) también influyen en la toma de decisión del modelo. Hemos visto que

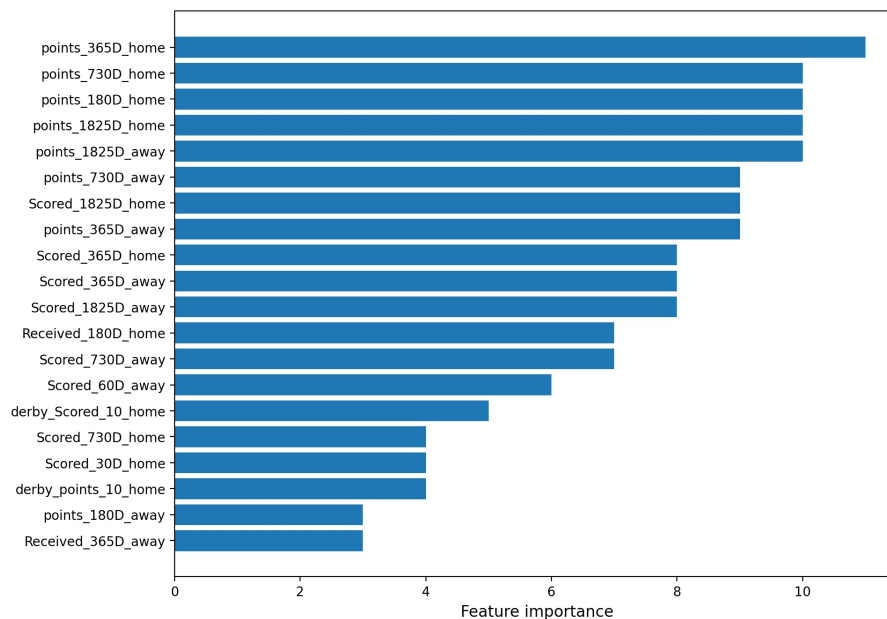


Figure 4.27: Feature importance in LightGBM.

el LightGBM puede obtener resultados realmente buenos en el accuracy, simplemente optimizando hiperparámetros y aplicando una selección de características apropiada. La gran limitación que hemos obtenido en los resultados es un pobre rendimiento en la predicción de empates, al igual que en los primeros experimentos realizados con ANN. Lo que no se puede poner en duda es el gran potencial de LightGBM para datos de tipo tabular.

Ahora que ya se han realizado experimentos con dos algoritmos de Machine Learning diferentes, vamos a analizar y resumir los resultados globales de esta serie de experimentos realizados. Además compararemos nuestros resultados con nuestro baseline, las cuotas de las casas de apuestas.

4.2.3. Conclusions

En este apartado vamos a recapitular los resultados obtenidos hasta ahora, tanto con las ANN como con LightGBM. Para sacar conclusiones sólidas y definitivas analizaremos los resultados al detalle, viendo dónde aciertan y fallan nuestros modelos. Hasta ahora hemos analizado los resultados en un nivel bastante general, salvo algunos partidos de ejemplo mostrados, ahora en cambio analizaremos a diferentes niveles la performance de nuestros modelos. Finalmente compararemos nuestros resultados y predicciones con las del Baseline.

Model \ League	Overall	ENG	SPA	GER	ITA	FRA
Baseline (odds)	0.545	0.54	0.54	0.52	0.55	0.5
ANN (event based)	0.5	0.53	0.5	0.52	0.47	0.46
ANN (hist short term)	0.52	0.55	0.54	0.51	0.5	0.48
ANN (uncalibrated)	0.545	0.59	0.52	0.54	0.54	0.55
ANN (calibrated)	0.521	0.54	0.53	0.49	0.53	0.52
LightGBM	0.532	0.57	0.53	0.57	0.53	0.47

Table 4.16: Accuracy in every national league for every algorithm. We show two models for ANN, one uncalibrated (with no balanced predictions) and other calibrated (with balanced predictions).

En la Table 4.16 se muestran los mejores resultados de cada tipo de modelo: ANN con el Event Based dataset, ANN con Historical Short term dataset, dos modelos ANN con Historical Long term dataset, uno con predicciones sin calibrar (poco acierto en empates) y otro con calibradas, y por último el modelo LightGBM. Vemos que sólo logramos alcanzar el acierto de las cuotas de las casas de apuestas con un modelo, el ANN sin calibrar.

Por otra parte en la misma tabla se muestran los respectivos accuracy scores de cada liga que aparece en el dataset. Vemos que aunque en cada liga se obtienen resultados similares a lo largo de los diferentes modelos, hay algunas excepciones. Vemos, pues, que por lo general la Premier League inglesa es la liga que mejor resultados obtenemos, y logramos superar el acierto del Baseline hasta en tres modelos, especialmente en el ANN uncalibrated donde superamos en un 5% el acierto de las casas de apuestas. La liga alemana y francesa también se logra superar al Baseline en dos modelos cada una, también hasta con un 5% más de acierto. Por otro lado, la liga española e italiana, pese a que no se obtienen resultados malos no se logra superar el acierto del Baseline. Pese a que la liga francesa sí se logra batir al Baseline con dos modelos, es la más complicada de predecir debido a su tremenda igualdad e irregularidad de sus equipos, lo cuál se demuestra con el acierto en los otros tres modelos.

Otro aspecto interesante a analizar es el acierto que se obtiene dependiendo el tramo de temporada en que se jueguen los partidos. El acierto a lo largo de la temporada suele ser bastante constante a lo largo de los modelos, habiendo generalmente diferencias de hasta 5-7% entre los diferentes modelos. Sin embargo hay excepciones, especialmente en meses en que se reanuda la competición tras meses parada, como en julio, agosto o septiembre, cuando empieza una nueva temporada tras 3 meses de parón. Esto afecta gravemente las predicciones en general, pero especialmente las de los modelos sin datos a largo plazo (Event Based and Historical Short Term). Vemos que el acierto en estos modelos mejora conforme va avanzando la competición y ya se han jugado varias jornadas.

Como es de esperar, nuestros modelos predicen más acertadamente los partidos entre

Month Model	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
Baseline (odds)	0.53	0.53	0.52	0.53	0.55	0.56	0.55	0.52	0.55	0.55	0.52	0.52
ANN (event based)	0.5	0.52	0.51	0.53	0.52	0.51	0.47	0.38	0.42	0.45	0.51	0.52
ANN (hist short term)	0.51	0.47	0.48	0.53	0.52	0.33	0.5	0.44	0.47	0.53	0.49	0.5
ANN (uncalibrated)	0.6	0.53	0.54	0.54	0.51	0.5	0.75	0.43	0.54	0.5	0.62	0.59
ANN (calibrated)	0.54	0.52	0.49	0.5	0.52	0.63	0.71	0.57	0.46	0.57	0.67	0.57
LightGBM	0.52	0.5	0.53	0.59	0.54	0.47	0.5	0.53	0.49	0.54	0.5	0.54

Table 4.17: Accuracy in month of the year for every algorithm. We show two models for ANN, one uncalibrated (with no balanced predictions) and other calibrated (with balanced predictions).

rivales de diferente nivel que entre rivales del mismo nivel en los que los partidos suelen ser igualados y ambos equipos llegan con estadísticas muy similares. En la Figure 4.28 hemos clasificado en diferentes clusters a los equipos en base a los puntos conseguidos durante el los últimos 5 y un año.

La clasificación la hemos realizado con un algoritmo de clustering llamado K-means con $k = 3$ clusters. El algoritmo K-means es un método de clustering que agrupa un conjunto de datos en K clusters. El proceso comienza eligiendo K centroides aleatorios, uno para cada cluster. Luego, se asigna cada punto de datos al cluster cuyo centroide está más cercano. Después de que todos los puntos de datos se han asignado, se recalcula el centroide de cada cluster y se repite el proceso de asignación hasta que los centroides ya no cambien significativamente o se alcance un número máximo de iteraciones. El resultado final es un conjunto de K clusters con centroides que representan las agrupaciones de datos. La elección de $k = 3$ ha sido práctica más que nada, ya que buscamos mostrar en un gráfico el acierto inter y intra cluster, y seleccionar un k mayor daría demasiadas combinaciones.

En nuestro caso, en verde (cluster 2) tenemos los mejores equipos, en naranja los equipos que suelen jugar europa pero sin ganar títulos y en azul los equipos de la parte media y baja tabla.

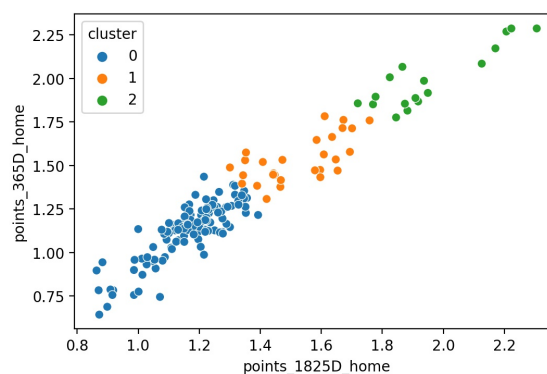


Figure 4.28: Clustering of the teams based on their points in the last 1 and 3 years.

Si cogemos las predicciones dadas por un modelo con F1-score en empates de 0.2 y calculamos las matrices de confusión y el acierto de los partidos que se han enfrentado entre los diferentes clusters (incluidos entre los equipos del mismo cluster), observamos (Figure 4.29) que la diferencia de acierto de un tipo de partido a otro es grande. En partidos con gran diferencia de nivel (cluster 2 vs cluster 0) el modelo roza el 80% de acierto, aunque es cierto que el modelo está sobreconfiando en sus predicciones y ignora todo

lo que no sea que gane el equipo favorito. Como ya hemos visto antes, las variables de largo plazo que representan el nivel histórico de un equipo pesan mucho en el modelo. Por otra parte, el modelo también sobreconfía en el equipo local cuando los dos equipos están igualados (cluster 0 vs 0 o 1 vs 1). Vemos que en los tipos de partidos que más falla el modelo es en los enfrentamientos entre dos muy buenos equipos (37% de accuracy), en los cuales hay una gran cantidad de empates, casi igual a victoria locales. En resumen, obtenemos mejores resultados en las combinaciones en las cuales hay menos empates, especialmente en la que hay más victorias locales.

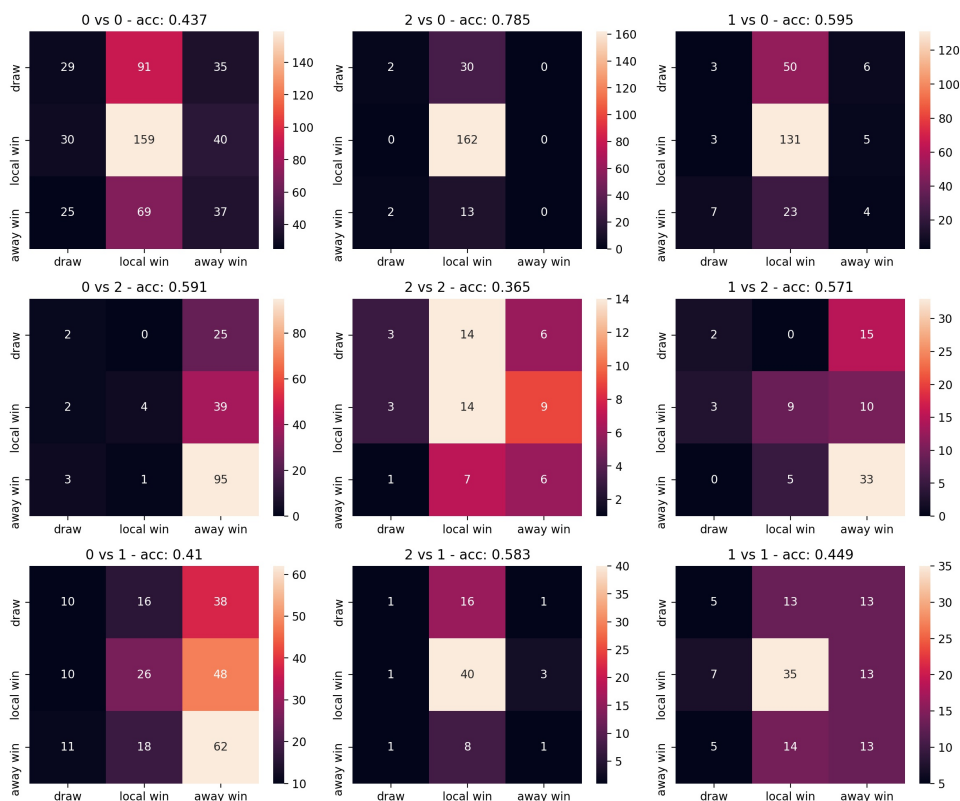
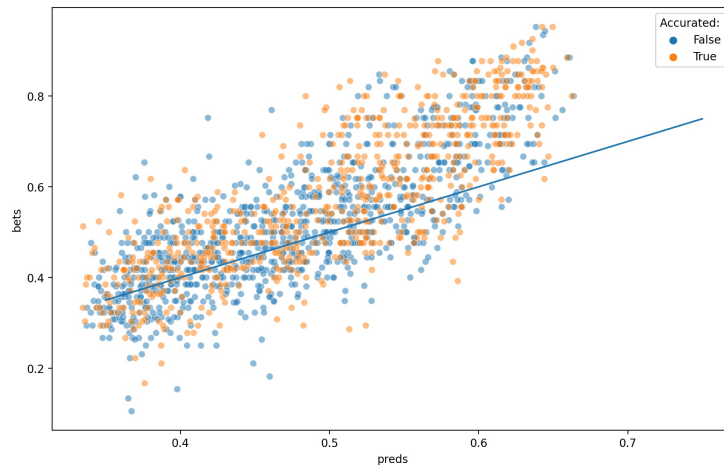


Figure 4.29: Test accuracy and confusion matrix of every type of match.

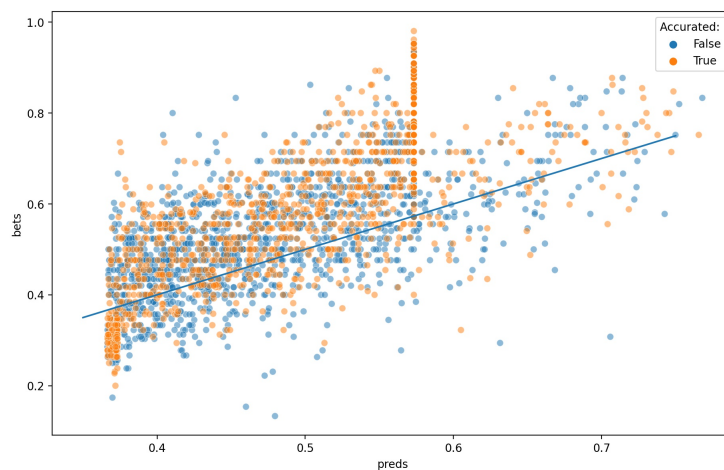
Dados estos aciertos en cada cluster, podemos concluir que es realmente complicado con los datos que disponíamos poder modelar mejor los partidos entre equipos similares en nivel. Sin embargo, podemos comparar nuestros resultados con las predicciones hechas por las casas de apuestas, las cuales cuentan con datos y herramientas realmente potentes para realizarlas. Con esta comparación veremos si nuestras predicciones son buenas y están al nivel o superan las de las casas de apuestas o, en cambio, están lejos de sus resultados.

Para realizar la comparativa (Figure 4.30), hemos cogido la predicción de nuestro modelo más alta de cada partido (sea empate, victoria local o visitante) y la comparamos con la misma predicción por parte de las casas de apuesta. Además mostramos en distinto color aquellas predicciones que se han acertado y las que no. Finalmente, se ha dibujado una línea que separa, por debajo las predicciones las cuales son mayores que las cuotas y por encima al contrario. Por lo tanto, si acertamos el resultado y el punto está por debajo de la línea significará que nuestro modelo le ha dado más probabilidad a ese resultado que la cuota de apuestas, y por tanto nuestro modelo habrá realizado una mejor predicción.

En la Figure 4.30 se muestran dos modelos, la de arriba muestra las predicciones del



(a) Best vs predictions of the model with highest accuracy (model with 54.5% accuracy).



(b) Best vs predictions of the model with highest f1-score in draws (model with $f = 10$).

Figure 4.30: Comparison of bets and predictions. Coloured in orange the well predicted matches and in blue the wrong predicted ones.

modelo ANN con mejor acierto y abajo el modelo con mejor acierto en empates. Vemos que la distribución de los puntos de la gráfica de abajo se ajusta más a la línea azul, es decir ambas predicciones son más parecidas (correlacionadas 1 a 1). En la gráfica de arriba se ve como el modelo realiza predicciones con probabilidades mucho más bajas que el otro modelo.

(%)	bet > pred.	pred. > bet
Accurate	30	11
Wrong	38	21

(a) Model with highest accuracy.

(%)	bet > pred.	pred. > bet
Accurate	26	16
Wrong	32	26

(b) Model with highest F1-score in draws.

Table 4.18: Difference and accuracy of bets and model predictions.

En la Table 4.18 se muestran los porcentajes de partidos que se aciertan y fallan en relación con la diferencia entre la cuota y la predicción del resultado pronosticado. Se muestran las predicciones del modelo con un f1-score de empates más grande. Vemos que en el modelo con mejor acierto en los empates acertamos el 16% de los partidos proponiendo una probabilidad mayor que la cuota de apuesta, un 5% más que el modelo

(mean)	bet > pred.	pred. > bet	(mean)	bet > pred.	pred. > bet
Accurate	0.51	0.418	Accurate	0.64	0.36
Wrong	0.482	0.428	Wrong	0.59	0.37

(a) Model probabilities.

(b) Betting probabilities.

Table 4.19: Mean probability of bets and model predictions.

con mejor acierto. Para ese 16% partidos nuestro modelo predice mejor los resultados y además acierta el resultado. Además la media de la probabilidad que da nuestro modelo para el resultado acertado en estos partidos es de tan sólo el 41.7% frente al 36% de media que dan las casas de apuestas (Figure 4.19). Es decir, además batimos las predicciones de la casa de apuestas en partidos igualados, donde ellos no tienen claro que va a pasar y ponen una cuota conservadora.

Pese a todo, ya hemos mencionado varias veces a lo largo de este trabajo que utilizar el output del modelo como probabilidades no es del todo preciso y que contienen un margen de error. Mostramos en Figure 4.20 cual sería el acierto en caso de considerar las probabilidades dentro de un Intervalo de Confianza del 95%.

Model \ CI (%)	acc.	acc. 95%	acc. 90%	acc. 99%
Baseline (odds)	0.545	0.504 - 0.552	0.473 - 0.590	0.523 - 0.545
ANN (event based)	0.5	0.402 - 0.541	0.390 - 0.570	0.490 - 0.510
ANN (hist short term)	0.52	0.410 - 0.551	0.421 - 0.580	0.516 - 0.525
ANN (uncalibrated)	0.545	0.500 - 0.590	0.460 - 0.625	0.534 - 0.552
ANN (calibrated)	0.521	0.440 - 0.594	0.383 - 0.653	0.510 - 0.540
LightGBM	0.532	0.500 - 0.552	0.467 - 0.584	0.526 - 0.536

Table 4.20: Best possible accuracy within different Confidence Intervals.

En la Table 4.20 se muestra el rango de acierto de cada modelo dentro de cada intervalo de confianza. Vemos que el ANN calibrated, aunque puede llegar a un 65% de acierto en un CI de 90%, es también el modelo más sensible y también puede caer a un mínimo de 38.3% de acierto. Esto es debido a que este modelo, con tal de calibrar las probabilidades (balancear las predicciones para que haya empates), las junta y hace predicciones menos agresivas. Esto demuestra que este modelo, pese a ser el mejor pronosticando los empates es también el menos fiable porque está muchas veces en el límite de fallar o acertar. Vemos, pues, que similar al baseline está el accuracy del ANN uncalibrated, que aunque ligeramente por debajo en el nivel inferior del CI supera en un 4% al baseline en el rango superior. Esto sugiere que el ANN uncalibrated es bastante robusto en las predicciones y además se queda muy cerca bastantes veces de acertar el resultado. El modelo de LightGBM, aunque por debajo en el accuracy también demuestra ser un modelo bastante robusto.

Con este análisis final comparativo de los diferentes modelos con el baseline damos por concluida la parte de experimentos. Hemos visto durante esta parte como nuestros modelos pueden igualar en acierto y error a las cuotas de la casas de apuestas, como nuestros modelos pueden en algunos casos predecir de una forma más robusta y contundente (con una probabilidad mayor) el resultado de un partido, y como podemos calibrar nuestras predicciones y ver lo fiables que son. Además hemos visto en qué tipo de partidos nue-

stros modelos lo hacen mejor (ligas, meses, nivel del partido...) y en qué ámbitos se puede sacar provecho de estas predicciones y en cuales tiene aún margen de mejora.

